TRƯỜNG ĐẠI HỌC SƯ PHẠM THÀNH PHỐ HỒ CHÍ MINH

KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN CUỐI KỲ

HỌC PHẦN: LẬP TRÌNH WINDOWS

ĐỀ TÀI: XÂY DỰNG HỆ THỐNG

QUẢN LÝ MƯỢN TRẢ THIẾT BỊ

Giảng viên hướng dẫn: ThS. Lê Thanh Thoại

Nhóm thực hiện: Nhóm KháBí

Đỗ Việt Hoàng – 51.01.104.031 (Nhóm trưởng)

Lê Hoàng Quân – 51.01.104.082

Ngô Minh Hải – 51.01.104.022

TP. Hồ Chí Minh, tháng 11 năm 2026

LỜI CẢM ƠN

Nhóm sinh viên thực hiện đề tài "Xây dựng hệ thống quản lý mượn trả thiết bị" xin gửi lời cảm ơn chân thành đến Trường Đại học Sư phạm Thành phố Hồ Chí Minh, Khoa Công nghệ Thông tin và giảng viên phụ trách học phần Lập trình Windows đã tạo điều kiện thuận lợi để nhóm vận dụng kiến thức lý thuyết vào việc phát triển một ứng dụng phần mềm có quy trình nghiệp vụ thực tế.

Nhóm đặc biệt cảm ơn Giảng viên hướng dẫn ThS. Lê Thanh Thoại đã tận tình giảng dạy, định hướng kiến trúc và cung cấp những kiến thức nền tảng quan trọng về lập trình ứng dụng Windows Forms, thiết kế cơ sở dữ liệu SQL Server, kiểm soát giao dịch (transaction) và tổ chức mã nguồn nhiều lớp. Quá trình thực hiện đồ án giúp các thành viên rèn luyện kỹ năng phân tích yêu cầu, kiểm soát tồn kho thiết bị, xử lý ngoại lệ và phối hợp làm việc nhóm.

Do giới hạn về thời gian, môi trường kiểm thử và kinh nghiệm triển khai thực tiễn, sản phẩm không tránh khỏi những điểm cần tiếp tục hoàn thiện. Nhóm rất mong nhận được các ý kiến đóng góp của giảng viên để nâng cao chất lượng và tính ứng dụng của hệ thống.

Xin chân thành cảm ơn!

GIỚI THIỆU THÀNH VIÊN VÀ PHÂN CÔNG

Bảng 0.1. Danh sách thành viên nhóm

STT	Họ và tên	Mã số sinh viên	Vai trò trong nhóm
1	Đỗ Việt Hoàng	51.01.104.031	Nhóm trưởng
2	Lê Hoàng Quân	51.01.104.082	Thành viên
3	Ngô Minh Hải	51.01.104.022	Thành viên

Công việc của nhóm được phân công theo thế mạnh của từng thành viên. Nhóm trưởng phụ trách giao diện và trải nghiệm người dùng, một thành viên phụ trách phần logic nghiệp vụ và định hướng kiến trúc, thành viên còn lại phụ trách kiểm thử và biên soạn báo cáo. Các thành viên thường xuyên trao đổi để thống nhất giao diện, nghiệp vụ và kịch bản kiểm thử trước khi hợp nhất vào mã nguồn chung.

Bảng 0.2. Phân công công việc và mức đóng góp

Thành viên	Vai trò	Hạng mục phụ trách chính	Mức đóng góp
Đỗ Việt Hoàng	Nhóm trưởng	Thiết kế giao diện và trải nghiệm người dùng (UI/UX): thư viện control `QuanLyMuonTra.UI`, các màn hình `Forms`/`Pages`, chế độ giao diện Sáng/Tối; điều phối công việc chung của nhóm.	100%
Lê Hoàng Quân	Thành viên	Xây dựng các hàm xử lý logic nghiệp vụ (`Services`, `Data`, transaction, kiểm soát tồn kho) và định hướng kiến trúc chung của hệ thống.	100%
Ngô Minh Hải	Thành viên	Kiểm thử phần mềm (thực thi 35 ca kiểm thử, chụp ảnh minh chứng) và biên soạn báo cáo tổng kết học phần.	100%

Mỗi thành viên đều hoàn thành 100% phần việc được giao trong hạng mục phụ trách của mình. Minh chứng đóng góp của từng thành viên được ghi nhận qua lịch sử commit và mục Contributors của repository GitHub của nhóm.

MỤC LỤC

LỜI CẢM ƠN ii

GIỚI THIỆU THÀNH VIÊN VÀ PHÂN CÔNG iii

DANH MỤC BẢNG BIỂU vii

DANH MỤC HÌNH ẢNH vii

1 TỔNG QUAN 1

1.1 Giới thiệu đề tài 1

1.2 Mục tiêu đề tài 1

1.3 Phạm vi nghiên cứu 2

1.4 Phương pháp thực hiện 3

2 CƠ SỞ LÝ THUYẾT 4

2.1 Tổng quan ngôn ngữ lập trình C# 4

2.2 WinForms, mô hình Page/Dialog và thư viện giao diện 4

2.3 LINQ – Truy vấn dữ liệu 4

2.4 Hệ quản trị cơ sở dữ liệu SQL Server 5

2.5 Kiến trúc nhiều lớp, ADO.NET và Dapper 5

2.6 Mã hóa mật khẩu và phân quyền động RBAC 6

2.7 Báo cáo tổng hợp, thống kê và in ấn biểu mẫu 6

3 THIẾT KẾ CHƯƠNG TRÌNH 7

3.1 Các chức năng chính 7

3.1.1 Đăng nhập hệ thống – F01 (`LoginForm.cs`) 7

3.1.2 Quản lý tài khoản và đặt lại mật khẩu – F04 (`UserPage.cs`) 7

3.1.3 Quản lý nhóm quyền và phân quyền – F05 (`RolePage.cs`) 7

3.1.4 Màn hình chính và Dashboard – F02 (`MainForm.cs`, `DashboardPage.cs`) 8

3.1.5 Quản lý loại thiết bị – F06 (`CategoryPage.cs`) 8

3.1.6 Quản lý thiết bị và tồn kho – F07 (DevicePage.cs) 8
3.1.7 Quản lý người mượn và lịch sử – F08 (BorrowerPage.cs) 8
3.1.8 Lập phiếu mượn nhiều thiết bị – F09 (LoanCreatePage.cs) . . . 9
3.1.9 Tra cứu và xem chi tiết phiếu mượn – F10, F11 (LoanSearchPage.cs) 9
3.1.10 Ghi nhận trả thiết bị theo tình trạng – F12 (ReturnCreatePage.cs) 9
3.1.11 Tra cứu, xem và hủy biên bản trả – F13 (ReturnRecordPage.cs) 9
3.1.12 Danh sách phiếu quá hạn – F14 (OverduePage.cs) 10
3.1.13 Báo cáo, thống kê top thiết bị và in ấn – F15, F18 (ReportPage.cs) 10
3.1.14 Cấu hình mã tự động và giao diện sáng/tối – F16 (CodeConfigPage.cs) 10
3.2 Thiết kế cơ sở dữ liệu chương trình 10
3.2.1 Sơ đồ quan hệ ERD 10
3.2.2 Mô tả chi tiết các bảng dữ liệu 11
3.2.3 View, Stored procedure và transaction 12
3.3 Kiến trúc và luồng xử lý 13
3.4 Quy tắc nghiệp vụ và kiểm soát tồn kho 15
3.5 Ma trận phân quyền người dùng 16

4 TRIỂN KHAI VÀ KẾT QUẢ 18
4.1 Môi trường phát triển 18
4.2 Cấu trúc Project 18
4.2.1 Sơ đồ tổng quan cấu trúc thư mục 18
4.2.2 Vai trò của các thành phần 19
4.3 Cấu hình và chạy chương trình 19
4.4 Chi tiết các chức năng quan trọng 20
4.4.1 Kiểm soát truy cập, băm mật khẩu và phân quyền động 20
4.4.2 Lập phiếu mượn nhiều thiết bị và trừ tồn kho có điều kiện 20
4.4.3 Ghi nhận trả theo tình trạng và hoàn tác khi hủy biên bản 21
4.4.4 Báo cáo tổng hợp, thống kê thiết bị mượn nhiều và in ấn 22
4.4.5 Sinh mã tự động an toàn đồng thời và chế độ sáng/tối 22
4.5 Hình ảnh sản phẩm thực tế 23
4.6 Kiểm thử và đánh giá kết quả 26

5 TỔNG KẾT **30**

5.1 Kết quả đạt được 30

5.2 Hạn chế 30

5.3 Hướng phát triển 31

5.4 Bài học kinh nghiệm 31

TÀI LIỆU THAM KHẢO **32**

PHỤ LỤC **34**

Phụ lục A. Liên kết và hồ sơ đồ án 34

Phụ lục B. Kịch bản trình diễn đề xuất 34

Phụ lục D. Mã nguồn tiêu biểu – Script tạo bảng dữ liệu 35

Phụ lục E. Mã nguồn tiêu biểu – Ràng buộc và chỉ mục 38

Phụ lục F. Mã nguồn tiêu biểu – View và Stored procedure 41

Phụ lục G. Mã nguồn tiêu biểu – Kết nối CSDL và transaction 44

Phụ lục H. Mã nguồn tiêu biểu – Lập phiếu mượn (tầng Data) 47

Phụ lục I. Mã nguồn tiêu biểu – Ghi nhận trả và hủy biên bản (tầng Data) 50

Phụ lục J. Mã nguồn tiêu biểu – Tầng nghiệp vụ (Services) 56

Phụ lục K. Mã nguồn tiêu biểu – Băm mật khẩu và phiên đăng nhập 59

DANH MỤC BẢNG BIỂU

0.1 Danh sách thành viên nhóm iii

0.2 Phân công công việc và mức đóng góp iii

1.1 Đối tượng sử dụng hệ thống 2

3.1 Tóm tắt 12 bảng dữ liệu chính 11

3.2 Các View và Stored Procedure trong cơ sở dữ liệu 13

3.3 Các tình huống kiểm soát nghiệp vụ 15

3.4 Ma trận phân quyền mặc định theo hai vai trò 16

4.1 Công nghệ và môi trường triển khai 18

4.2 Kết quả thực thi 35 kịch bản kiểm thử 26

DANH MỤC HÌNH ẢNH

3.1 Sơ đồ quan hệ thực thể (ERD) hệ thống Quản lý mượn trả thiết bị 11
3.2 Mô hình kiến trúc MVP (Model – View – Presenter), SQL Server và thư viện UI 14
3.3 Quy trình nghiệp vụ từ lập phiếu mượn đến ghi nhận trả và in biên bản 14

4.1 Giao diện đăng nhập và thông báo kiểm tra tài khoản/mật khẩu 20
4.2 Giao diện lập phiếu mượn nhiều thiết bị và giỏ thiết bị (F09) 21
4.3 Hộp thoại ghi nhận trả thiết bị theo tình trạng (F12) 21
4.4 Cửa sổ xem trước bản in phiếu mượn và biên bản bàn giao (F15) 22
4.5 Màn hình báo cáo tổng hợp và thống kê thiết bị mượn nhiều (F15, F18) 22
4.6 Màn hình cấu hình quy tắc sinh mã tự động và xem trước mã (F16) 23
4.7 Giao diện chính khi chuyển đổi chế độ Sáng / Tối (ThemeMode) 23
4.8 Màn hình Tổng quan (Dashboard F02) của tài khoản Nhân viên kho 24
4.9 Màn hình Tổng quan (Dashboard F02) của tài khoản Người quản lý 24
4.10 Giao diện quản lý danh mục thiết bị và lọc tồn kho (F07) 24
4.11 Giao diện quản lý loại thiết bị (F06) 25
4.12 Giao diện quản lý hồ sơ người mượn và lịch sử mượn (F08) 25
4.13 Giao diện tra cứu phiếu mượn và hộp thoại chi tiết phiếu (F10, F11) 25
4.14 Giao diện quản lý biên bản trả và danh sách phiếu quá hạn (F13, F14) 26
4.15 Giao diện quản lý người dùng và phân quyền nhóm quyền (F04, F05) 26

CHƯƠNG 1

TỔNG QUAN

1.1. Giới thiệu đề tài

Trong công tác quản lý tài sản, vật tư và trang thiết bị dùng chung tại các trường học, phòng thí nghiệm, trung tâm đào tạo hoặc doanh nghiệp (như máy tính xách tay, máy chiếu, máy ảnh, micro, thiết bị mạng), hoạt động mượn và trả thiết bị diễn ra thường xuyên mỗi ngày. Nếu quản lý thủ công bằng sổ giấy hoặc các tệp bảng tính rời rạc, nhân viên kho rất dễ gặp tình trạng không nắm rõ số lượng khả dụng thực tế, cho mượn vượt tồn kho, khó theo dõi các phiếu trả nhiều đợt (trả một phần), thất lạc thông tin thiết bị hư hỏng/mất hoặc phát hiện trễ các phiếu mượn quá hạn.

Đề tài "Xây dựng hệ thống quản lý mượn trả thiết bị" tập trung thiết kế và phát triển ứng dụng desktop chuyên biệt dành cho nhân viên kho và người quản lý. Phần mềm được xây dựng trên nền tảng C# Windows Forms (.NET 10), sử dụng hệ quản trị cơ sở dữ liệu Microsoft SQL Server kết hợp thư viện truy cập dữ liệu Dapper và tổ chức mã nguồn theo kiến trúc phân lớp theo hướng MVP (Model – View – Presenter): View (Forms/Pages và thư viện UI), Presenter (Services), Model (Models và Data), cùng lớp Security hỗ trợ phiên đăng nhập và phân quyền. Bên cạnh bộ thư viện control tùy biến `QuanLyMuonTra.UI` hỗ trợ giao diện Sáng/Tối (Light/Dark Mode), hệ thống cung cấp cơ chế phân quyền động theo chức năng (RBAC), sinh mã tự động an toàn đồng thời và in ấn phiếu mượn/biên bản bàn giao trực tiếp.

1.2. Mục tiêu đề tài

- Xây dựng giao diện Windows Forms hiện đại, đồng nhất với hệ thống control bo góc, thẻ thống kê (StatCard), bảng dữ liệu có nhãn trạng thái (Badge) và hỗ trợ chuyển đổi chủ đề Sáng/Tối.
- Thiết kế cơ sở dữ liệu quan hệ chuẩn hóa gồm 12 bảng, 1 view tổng hợp và 3 stored procedure, tích hợp đầy đủ khóa chính, khóa ngoại, chỉ mục (index) và ràng buộc toàn vẹn (CHECK/UNIQUE).
- Kiểm soát chặt chẽ nghiệp vụ mượn trả bằng giao dịch SQL (Transaction) và cơ chế khóa bản ghi (UPDLOCK, HOLDLOCK), đảm bảo tồn kho khả dụng không bao giờ âm ngay cả khi nhiều nhân viên thao tác đồng thời.

- Hỗ trợ trả thiết bị linh hoạt (trả đủ hoặc trả một phần qua nhiều đợt), phân loại chính xác 4 tình trạng khi trả (Tốt, Hư hỏng, Mất, Thiếu phụ kiện) để tự động điều chuyển nhóm tồn kho tương ứng.
- Bảo mật thông tin đăng nhập bằng thuật toán băm mật khẩu PBKDF2-SHA256 kèm salt ngẫu nhiên; triển khai phân quyền động theo vai trò (Role-Based Access Control) kiểm tra kép ở cả giao diện và tầng nghiệp vụ.
- Cung cấp công cụ theo dõi phiếu quá hạn, báo cáo tổng hợp mượn trả – tồn kho, xếp hạng thiết bị được mượn nhiều và xem trước/in phiếu mượn, biên bản bàn giao.

1.3. Phạm vi nghiên cứu

Phạm vi của hệ thống bao gồm 15 nhóm chức năng cốt lõi được đánh mã từ F01 đến F16 và F18 (trừ F03, F17 không nằm trong phạm vi): Đăng nhập và xác thực (F01), Bảng điều khiển Dashboard (F02), Quản lý người dùng (F04), Quản lý nhóm quyền và phân quyền (F05), Quản lý loại thiết bị (F06), Quản lý thiết bị (F07), Quản lý người mượn và lịch sử mượn (F08), Lập phiếu mượn nhiều thiết bị (F09), Tra cứu phiếu mượn (F10), Xem chi tiết phiếu mượn và lịch sử trả (F11), Ghi nhận trả thiết bị (F12), Quản lý và hủy biên bản trả (F13), Theo dõi phiếu quá hạn (F14), Báo cáo tổng hợp và in ấn (F15), Cấu hình mã tự động (F16) và Thống kê thiết bị mượn nhiều (F18).

Những nội dung chưa nằm trong phiên bản hiện tại gồm: cổng web/ứng dụng di động cho người mượn tự đăng ký trực tuyến, tích hợp quét mã vạch/RFID phần cứng, gửi email/SMS nhắc hạn tự động, quản lý chi phí đền bù/sửa chữa tài chính và đồng bộ đa chi nhánh đám mây. Đây là các định hướng mở rộng trong tương lai.

Bảng 1.1. Đối tượng sử dụng hệ thống

Nhóm đối tượng	**Vai trò và nhu cầu sử dụng chính**
Nhân viên kho	Thực hiện nghiệp vụ hằng ngày: xem Dashboard, tra cứu loại thiết bị, quản lý thiết bị và người mượn, lập phiếu mượn, ghi nhận trả, xem biên bản bàn giao, theo dõi phiếu quá hạn và in phiếu/biên bản.
Người quản lý	Toàn quyền nghiệp vụ của nhân viên kho; quản lý tài khoản người dùng, thiết lập nhóm quyền và gán 17 quyền chức năng, hủy biên bản trả sai sót, cấu hình quy tắc sinh mã tự động và xem báo cáo/thống kê tổng hợp.
Người mượn thiết bị	Cán bộ, giảng viên, sinh viên hoặc nhân viên các phòng ban có hồ sơ trong bảng Borrowers; nhận thiết bị, bàn giao trả và ký xác nhận trên phiếu mượn/biên bản in giấy (không trực tiếp đăng nhập phần mềm).

1.4. Phương pháp thực hiện

Nhóm áp dụng quy trình phát triển phần mềm theo 5 giai đoạn tăng trưởng (GĐ1 đến GĐ5) được ghi nhận chi tiết trong `CHANGELOG.md`. Trước tiên, nhóm khảo sát bài toán quản lý thiết bị, chuẩn hóa lược đồ cơ sở dữ liệu và xây dựng bộ khung giao diện dùng chung (`QuanLyMuonTra.UI`). Tiếp theo, nhóm lần lượt hiện thực hóa phân hệ danh mục, phân hệ lập phiếu mượn, phân hệ trả thiết bị – biên bản – quá hạn, và cuối cùng là phân hệ quản trị – báo cáo – in ấn. Toàn bộ quy tắc nghiệp vụ được rà soát tĩnh và đối chiếu với danh sách 35 ca kiểm thử tiêu chuẩn trong `docs/TEST_CASES.md` [9], [10].

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT

2.1. Tổng quan ngôn ngữ lập trình C#

C# là ngôn ngữ lập trình hướng đối tượng, kiểu mạnh (strongly-typed) và hiện đại thuộc nền tảng .NET của Microsoft. Trong dự án này, C# trên .NET 10 (`net10.0-windows`) được khai thác cùng các tính năng ngôn ngữ mới như `record` bất biến (`NavItem`, `Created`, `Auth Result`), biểu thức `switch` đối sánh mẫu (pattern matching), `Nullable` reference types, `ImplicitUsings`, `GeneratedRegex` biên dịch sẵn lúc build và lập trình bất đồng bộ `async/await` kết hợp `Task.Run` để giữ giao diện mượt mà khi truy vấn cơ sở dữ liệu [2].

2.2. WinForms, mô hình Page/Dialog và thư viện giao diện

Windows Forms (WinForms) là framework xây dựng ứng dụng desktop dựa trên cơ chế hướng sự kiện (event-driven) và kỹ thuật vẽ đồ họa GDI+ (`System.Drawing`, `System.Drawing.Drawing2D`). Thay vì mở hàng loạt cửa sổ rời rạc gây rối mắt, ứng dụng `QuanLyMuonTra` tổ chức theo mô hình khung điều hướng tập trung (Single-Shell Navigation): cửa sổ chính `MainForm` đóng vai trò khung chứa thanh menu bên trái (`pnlNav`), thanh tiêu đề phía trên và vùng nội dung động (`pnlContent`) nạp các `UserControl` chức năng (`Pages`) tương ứng [1].

Đặc biệt, nhóm tự xây dựng project thư viện giao diện riêng mang tên `QuanLyMuonTra.UI` hoàn toàn không phụ thuộc vào các bộ thư viện thương mại bên thứ ba (như DevExpress hay Guna UI). Thư viện này cung cấp các lớp cơ sở `ThemedForm`, `ThemedUserControl`, `ThemedPanel`, `CardPanel` bo góc chống răng cưa (`SmoothingMode.AntiAlias`), nút bấm `AppButton` 3 trạng thái, ô nhập liệu `AppTextBox` có biểu tượng `Segoe MDL2 Assets`, thẻ chỉ số `StatCard` và bộ định dạng lưới `AppGrid` hỗ trợ vẽ nhãn màu trạng thái (`EnableBadges`).

2.3. LINQ – Truy vấn dữ liệu

Language Integrated Query (LINQ) cung cấp cú pháp truy vấn dữ liệu thống nhất ngay trong ngôn ngữ C# thông qua các toán tử mở rộng như `Where`, `Select`, `GroupBy`, `OrderBy`, `Sum`, `Any`, `All`, `FirstOrDefault` và `ToDictionary` [3].

Đối chiếu mã nguồn thực tế của dự án: hệ thống sử dụng mạnh mẽ LINQ to Objects để xử lý tập dữ liệu trong bộ nhớ tại tầng nghiệp vụ và giao diện. Ví dụ tiêu biểu gồm: gộp các dòng thiết bị trùng mã và sắp xếp theo thứ tự `DeviceId` tăng dần nhằm chống deadlock khi lập phiếu (`s.Items.GroupBy(i => i.DeviceId).OrderBy(g => g.Key)` trong `LoanRepository.cs`), tổng hợp số lượng thiết bị trả theo từng nhóm tình trạng tồn kho (`ReturnRepository.cs`), lọc menu điều hướng theo quyền người dùng (`Nav Catalog.Items.Where(i => Session.Has(i.Permission))`) và lọc trạng thái tồn kho trên `DeviceRepository.Search`. Đối với truy vấn cơ sở dữ liệu SQL Server, dự án sử dụng Dapper kết hợp câu lệnh SQL tham số hóa và Stored Procedure thay vì dùng Entity Framework (`LINQ to Entities`).

2.4. Hệ quản trị cơ sở dữ liệu SQL Server

Microsoft SQL Server (từ phiên bản 2017 trở lên) được sử dụng để lưu trữ và bảo toàn dữ liệu cho hệ thống. Cơ sở dữ liệu `QuanLyMuonTra` khai thác sâu các cơ chế toàn vẹn của hệ quản trị quan hệ: khóa chính tự tăng (`IDENTITY`), khóa ngoại (`FOREIGN KEY`), chỉ mục duy nhất (`UNIQUE INDEX`) và các ràng buộc kiểm tra miền giá trị (`CHECK CONSTRAINT`) như bảo đảm tổng số lượng thiết bị khả dụng, hư hỏng và mất không vượt quá tổng số lượng đầu tư (`CK_Devices_Inventory`) [5].

Ngoài ra, hệ thống sử dụng các hàm và cú pháp hiện đại của T-SQL như `CREATE OR ALTER`, `STRING_AGG` (gộp chuỗi tên các thiết bị chưa trả trong thủ tục quá hạn), toán tử `CROSS APPLY` / `OUTER APPLY`, biểu thức bảng chung `CTE` (`WITH onloan AS ...`) và gợi ý khóa dòng `WITH (UPDLOCK, HOLDLOCK)` để kiểm soát truy cập đồng thời.

2.5. Kiến trúc nhiều lớp, ADO.NET và Dapper

Chương trình được thiết kế theo kiến trúc phân lớp hướng MVP (Model – View – Presenter) nhằm tách biệt trách nhiệm giữa hiển thị, xử lý trung gian và lưu trữ dữ liệu. Thành phần View gồm `Forms`, `Pages` và thư viện `QuanLyMuonTra.UI`; Presenter là `Services`; Model gồm `Models` và `Data`. View (`Forms`, `Pages`) chỉ phụ thuộc vào Presenter (`Services`) và `Models`, tuyệt đối không viết câu lệnh SQL trực tiếp trên Form. Presenter (`Services`) sử dụng lớp tiện ích trung tâm `Svc.Run` để kiểm tra phiên đăng nhập, xác thực quyền hạn và bắt mọi ngoại lệ chuyển thành cấu trúc `Result<T>` an toàn [4], [6].

Ở tầng truy cập dữ liệu (`Data`), dự án kết hợp `Microsoft.Data.SqlClient` (ADO.NET) với Micro-ORM **Dapper** [8]. Dapper hỗ trợ ánh xạ nhanh kết quả truy vấn SQL sang danh sách đối tượng C# (`Query<T>`, `QuerySingleOrDefault<T>`, `Execute Scalar<T>`) và tự động gắn tham số chống tấn công SQL Injection. Quản lý giao dịch được

tập trung tại phương thức `Db.InTransaction`, sử dụng mức cô lập `IsolationLevel.ReadCommitted` và tự động thử lại (retry) tối đa 3 lần với độ trễ tăng dần nếu gặp lỗi SQL Server Deadlock (mã lỗi `1205`).

2.6. Mã hóa mật khẩu và phân quyền động RBAC

Để bảo vệ mật khẩu người dùng, lớp `PasswordHasher` sử dụng thuật toán dẫn xuất khóa chuẩn công nghiệp **PBKDF2-SHA256** (`Rfc2898DeriveBytes.Pbkdf2`) với 100.000 vòng lặp (`Iterations = 100_000`), chuỗi muối ngẫu nhiên 16 byte (`SaltSize = 16`) sinh từ `RandomNumberGenerator` và độ dài khóa đầu ra 32 byte. Chuỗi lưu trong cột `PasswordHash` có định dạng `PBKDF2$100000$<base64_salt>$<base64_hash>`. Khi đối chiếu mật khẩu đăng nhập, hệ thống dùng hàm `CryptographicOperations.FixedTimeEquals` để so sánh mảng byte trong thời gian hằng số, ngăn chặn tấn công đo thời gian (timing attack) [7].

Khác với cách phân quyền cứng theo tên vai trò, dự án triển khai mô hình **Phân quyền dựa trên vai trò động (Dynamic RBAC)** thông qua 4 bảng dữ liệu: `Users`, `Roles`, `Permissions` và `RolePermissions`. Hệ thống định nghĩa 17 mã quyền chi tiết trong lớp tĩnh `Perm`. Khi người dùng đăng nhập, tập quyền của vai trò được nạp vào `Session` (`HashSet<string>`). Mọi thao tác vừa được ẩn/hiện thông minh trên giao diện, vừa được kiểm tra bắt buộc tại tầng `Services` qua `Svc.Run`.

2.7. Báo cáo tổng hợp, thống kê và in ấn biểu mẫu

Hệ thống cung cấp công cụ tổng hợp số liệu mượn trả theo khoảng thời gian và thống kê tồn kho thời gian thực tại `ReportPage` (F15), đồng thời xếp hạng các thiết bị có tần suất mượn cao nhất tại `PopularDevicePage` (F18). Đối với nhu cầu xuất chứng từ giấy để ký nhận, lớp `PrintHelper` sử dụng trực tiếp thành phần `PrintDocument` và `PrintPreviewDialog` của .NET Windows Forms để dàn trang và vẽ biểu mẫu **Phiếu mượn thiết bị** cũng như **Biên bản trả / bàn giao thiết bị** kèm đầy đủ khu vực chữ ký xác nhận của các bên liên quan mà không phụ thuộc vào thư viện báo cáo bên ngoài.

CHƯƠNG 3

THIẾT KẾ CHƯƠNG TRÌNH

3.1. Các chức năng chính

3.1.1. Đăng nhập hệ thống – F01 (`LoginForm.cs`)

Màn hình đăng nhập không viền (borderless form) hỗ trợ kéo thả cửa sổ qua P/Invoke `user32.dll` (`EnableDrag`). Khi người dùng nhập tên đăng nhập và mật khẩu, sự kiện `Btn Login_Click` kiểm tra dữ liệu rỗng, khóa tạm nút bấm (`Đang đăng nhập...`) và gọi bất đồng bộ `UserRepository.Authenticate`. Hệ thống kiểm tra tài khoản tồn tại, đối chiếu mật khẩu băm qua `PasswordHasher.Verify`, kiểm tra cờ hoạt động `IsActive = 1`, sau đó nạp danh sách mã quyền từ bảng `RolePermissions` vào `Session.Start` trước khi mở `MainForm`.

3.1.2. Quản lý tài khoản và đặt lại mật khẩu – F04 (`UserPage.cs`)

Dành cho người dùng có quyền `USER_MANAGE`. Cho phép tìm kiếm tài khoản theo tên đăng nhập, họ tên hoặc email; thêm mới tài khoản (`UserEditDialog`), cập nhật thông tin cá nhân và nhóm quyền, khóa hoặc mở khóa tài khoản (`SetUserActive`) và đặt lại mật khẩu (`PasswordResetDialog`). Presenter `AdminService` kiểm tra độ dài tên đăng nhập (3–50 ký tự), mật khẩu tối thiểu 8 ký tự, và chặn thao tác tự khóa chính tài khoản đang đăng nhập (`id == currentUserId`).

3.1.3. Quản lý nhóm quyền và phân quyền – F05 (`RolePage.cs`)

Dành cho người quản lý có quyền `ROLE_MANAGE`. Giao diện chia hai cột bằng `Split Container`: cột trái hiển thị danh sách nhóm quyền (`Roles`) kèm số lượng tài khoản thuộc nhóm, cho phép thêm mới hoặc đổi tên/mô tả nhóm quyền (`RoleEditDialog`); cột phải hiển thị danh sách 17 quyền chức năng dưới dạng `CheckedListBox`. Khi nhấn `Lưu quyền`, phương thức `AdminRepository.SaveRolePermissions` thực thi trong một transaction để xóa tập quyền cũ và chèn tập quyền mới cho nhóm được chọn.

3.1.4. Màn hình chính và Dashboard – F02 (`MainForm.cs`, `DashboardPage.cs`)

Khung chính `MainForm` tự động dựng thanh điều hướng bên trái (`BuildNav`) dựa trên danh mục `NavCatalog.Items`, chỉ hiển thị những mục mà phiên làm việc hiện tại có quyền truy cập, được gom theo 4 phân nhóm: Tổng quan, Danh mục, Mượn trả, Báo cáo và Hệ thống. Trang mặc định `DashboardPage` (F02) hiển thị 4 thẻ chỉ số trực quan (`StatCard`) gồm: Tổng số đầu thiết bị đang hoạt động, Tổng số đơn vị khả dụng, Số phiếu đang mượn và Số phiếu quá hạn; phía dưới là bảng 8 phiếu mượn gần nhất kèm nhãn màu trạng thái.

3.1.5. Quản lý loại thiết bị – F06 (`CategoryPage.cs`)

Hiển thị danh mục phân loại thiết bị (như `LAPTOP`, `PROJ`, `CAM`, `AUDIO`, `NET`, `OTHER`) kèm số lượng thiết bị đang hoạt động thuộc mỗi loại. Nhân viên kho chỉ có quyền `CATEGORY_VIEW` nên các nút Thêm, Sửa, Xóa tự động ẩn trên giao diện. Người quản lý có quyền `CATEGORY_MANAGE` được phép thêm/sửa loại thiết bị (mã loại tự động chuẩn hóa chữ in hoa, chỉ nhận chữ không dấu, số, gạch ngang/dưới) và chỉ xóa được loại thiết bị khi chưa có thiết bị nào tham chiếu tới.

3.1.6. Quản lý thiết bị và tồn kho – F07 (`DevicePage.cs`)

Quản lý thông tin chi tiết từng thiết bị: mã thiết bị, tên thiết bị, loại thiết bị, đơn vị tính, tổng số lượng (`TotalQuantity`), khả dụng (`AvailableQuantity`), đang mượn (`OnLoanQuantity`), hư hỏng (`DamagedQuantity`) và mất (`LostQuantity`). Hỗ trợ tìm kiếm có trễ (debounce timer 350ms) và lọc theo loại thiết bị. Khi thêm mới, hệ thống gợi ý mã tự động (`TB0009...`). Khi cập nhật tổng số lượng, hệ thống tự động cộng/trừ phần chênh lệch vào số lượng khả dụng và ngăn giảm tổng số lượng xuống thấp hơn tổng số đang mượn cộng hư hỏng/mất. Ngoài ra, `DeviceService.RecoverDamaged` hỗ trợ chuyển số lượng thiết bị hư hỏng đã sửa chữa xong quay về nhóm khả dụng.

3.1.7. Quản lý người mượn và lịch sử – F08 (`BorrowerPage.cs`)

Lưu trữ hồ sơ người mượn (`BorrowerCode`, họ tên, điện thoại, email, phòng ban/đơn vị, địa chỉ, ghi chú và trạng thái hoạt động). `BorrowerService` kiểm tra định dạng số điện thoại và email bằng biểu thức chính quy biên dịch sẵn (`GeneratedRegex`). Người dùng có thể lọc xem cả người mượn đã ngừng hoạt động, mở hộp thoại `BorrowerHistoryDialog` để tra cứu toàn bộ lịch sử phiếu mượn của người đó, hoặc xóa hồ sơ nếu người mượn chưa từng phát sinh phiếu mượn nào.

3.1.8. Lập phiếu mượn nhiều thiết bị – F09 (`LoanCreatePage.cs`)

Màn hình chia đôi gồm danh sách thiết bị còn khả dụng bên trái và giỏ thiết bị chọn mượn (`CartRow`) bên phải. Nhân viên chọn người mượn, ngày mượn, hạn dự kiến trả và thêm một hoặc nhiều thiết bị vào phiếu; nếu chọn trùng thiết bị đã có trong giỏ thì hệ thống tự động cộng dồn số lượng và đối chiếu không vượt quá `AvailableQuantity`. Khi nhấn `Lưu phiếu mượn`, `LoanService.Create` thực thi trong một SQL Transaction: kiểm tra người mượn đang hoạt động, gọi `sp_GetNextCode` sinh mã phiếu (`PM-yyyyMMdd-xxxx`), tạo đầu phiếu `LoanSlips`, trừ tồn kho có điều kiện và lưu từng dòng `LoanSlipDetails`.

3.1.9. Tra cứu và xem chi tiết phiếu mượn – F10, F11 (`LoanSearchPage.cs`)

Cho phép tìm kiếm phiếu mượn theo từ khóa (mã phiếu, tên/mã người mượn, số điện thoại), lọc theo khoảng ngày mượn, khoảng hạn trả và 4 trạng thái hiển thị (`Đang mượn`, `Trả một phần`, `Đã trả`, `Quá hạn`). Khi nhấp đúp hoặc nhấn `Xem chi tiết`, hộp thoại `LoanSlipDetailDialog` (F11) hiển thị đầy đủ thông tin đầu phiếu và 2 tab: danh sách thiết bị trong phiếu (số lượng mượn, đã trả, còn lại) và lịch sử tất cả các biên bản trả của phiếu đó.

3.1.10. Ghi nhận trả thiết bị theo tình trạng – F12 (`ReturnCreatePage.cs`)

Liệt kê danh sách các phiếu mượn còn số lượng thiết bị chưa trả (`RemainingQty > 0`). Khi chọn một phiếu và mở `ReturnCreateDialog`, nhân viên nhập ngày trả, tên người giao và điền số lượng trả cho từng thiết bị kèm tình trạng bàn giao (`Tốt`, `Hư hỏng`, `Mất`, `Thiếu phụ kiện`) và ghi chú. Nếu chọn tình trạng `Hư hỏng` hoặc `Thiếu phụ kiện`, hệ thống bắt buộc phải nhập ghi chú mô tả chi tiết trước khi lưu.

3.1.11. Tra cứu, xem và hủy biên bản trả – F13 (`ReturnRecordPage.cs`)

Quản lý danh sách biên bản bàn giao trả thiết bị (`BB-yyyyMMdd-xxxx`), lọc theo từ khóa, khoảng ngày trả và trạng thái (`Hiệu lực` / `Đã hủy`). Hộp thoại `ReturnRecord DetailDialog` hiển thị chi tiết từng dòng thiết bị đã trả và cho phép xem trước/in biên bản. Đối với tài khoản có quyền `RECORD_CANCEL`, nút `Hủy biên bản` cho phép nhập lý do hủy để hoàn tác số lượng đã trả trên phiếu mượn và trừ ngược lại đúng nhóm tồn kho đã cộng trước đó mà không xóa cứng bản ghi.

3.1.12. Danh sách phiếu quá hạn – F14 (`OverduePage.cs`)

Gọi thủ tục lưu trữ `dbo.sp_GetOverdueSlips` để truy xuất các phiếu mượn đã quá ngày hạn trả (`DueDate < AsOfDate`) nhưng vẫn còn thiết bị chưa trả đủ. Màn hình hỗ trợ tìm kiếm theo mã phiếu, tên hoặc số điện thoại người mượn, lọc theo số ngày trễ tối thiểu, đồng thời hiển thị cột tổng hợp danh sách tên và số lượng từng thiết bị đang nợ (`Unreturned Devices`).

3.1.13. Báo cáo, thống kê top thiết bị và in ấn – F15, F18 (`ReportPage.cs`)

Trang `ReportPage` (F15) thống kê theo khoảng ngày: tổng số phiếu mượn, tổng số lượng mượn, đã trả, còn phải trả, số phiếu quá hạn và bức tranh tồn kho hiện tại (số đầu thiết bị hoạt động, khả dụng, hư hỏng, mất). Hệ thống phân tách rõ quyền `REPORT_PRINT` (chỉ được in phiếu/biên bản) và quyền `REPORT_VIEW` (được xem số liệu báo cáo tổng hợp). Trang `PopularDevicePage` (F18) xếp hạng Top N thiết bị được mượn nhiều nhất theo tổng số lượng mượn giảm dần.

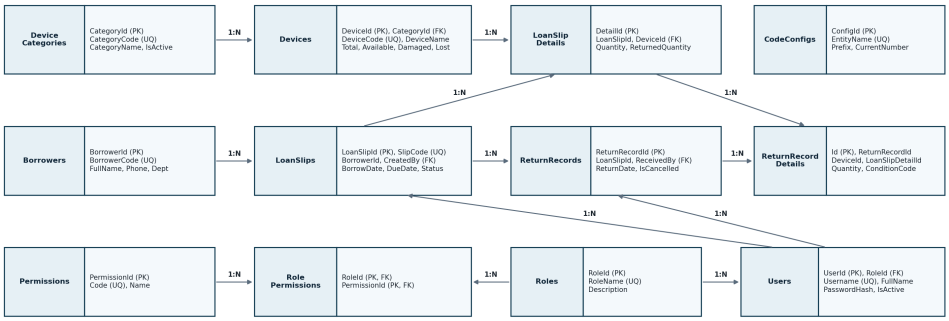
3.1.14. Cấu hình mã tự động và giao diện sáng/tối – F16 (`CodeConfigPage.cs`)

Trang `CodeConfigPage` (F16) cho phép người quản lý cấu hình quy tắc sinh mã cho 4 thực thể (`LoanSlip`, `ReturnRecord`, `Device`, `Borrower`) gồm: tiền tố (`Prefix`), độ dài phần số (`NumberLength` từ 1–9), tùy chọn chèn ngày (`UseDate`) và đặt lại bộ đếm hằng ngày (`ResetDaily`), kèm nút `Xem mã tiếp theo` (`Peek`). Ngoài ra, nút chuyển đổi giao diện Sáng/Tối trên thanh tiêu đề `MainForm` gọi `Theme.Toggle()` giúp toàn bộ Form và control tự động vẽ lại bảng màu tức thời.

3.2. Thiết kế cơ sở dữ liệu chương trình

3.2.1. Sơ đồ quan hệ ERD

Cơ sở dữ liệu `QuanLyMuonTra` được thiết kế gồm 12 bảng dữ liệu chuẩn hóa, chia thành 3 phân khu: Phân quyền & Người dùng (`Roles`, `Permissions`, `RolePermissions`, `Users`), Danh mục & Cấu hình (`DeviceCategories`, `Devices`, `Borrowers`, `CodeConfigs`) và Giao dịch Mượn – Trả (`LoanSlips`, `LoanSlipDetails`, `Return Records`, `ReturnRecordDetails`). Hình 3.1 dưới đây biểu diễn sơ đồ quan hệ thực thể được trích xuất trực tiếp từ các tệp `Database/02_CreateTables.sql` và `03_ Constraints.sql`.



Hình 3.1. Sơ đồ quan hệ thực thể (ERD) hệ thống Quản lý mượn trả thiết bị

3.2.2. Mô tả chi tiết các bảng dữ liệu

Bảng 3.1. Tóm tắt 12 bảng dữ liệu chính

Bảng	Khóa chính	Trường tiêu biểu	Ý nghĩa
Roles	RoleId	RoleName (UQ), Description	Lưu danh sách vai trò/nhóm quyền trong hệ thống.
Permissions	PermissionId	Code (UQ), Name	Danh mục 17 mã quyền chức năng chuẩn của phần mềm.
RolePermissions	(RoleId, PermissionId)	RoleId (FK), PermissionId (FK)	Bảng trung gian ánh xạ quan hệ nhiều–nhiều giữa vai trò và quyền.
Users	UserId	Username (UQ), PasswordHash, FullName, RoleId (FK), IsActive	Lưu tài khoản nhân viên kho và người quản lý; mật khẩu băm PBKDF2.
DeviceCategories	CategoryId	CategoryCode (UQ), CategoryName, Description, IsActive	Danh mục phân loại trang thiết bị (Laptop, Máy chiếu, Âm thanh...).
Devices	DeviceId	DeviceCode (UQ), DeviceName, CategoryId (FK), TotalQuantity, AvailableQuantity, DamagedQuantity, LostQuantity	Quản lý thiết bị và cân bằng 4 nhóm số lượng tồn kho.
Borrowers	BorrowerId	BorrowerCode (UQ), FullName, Phone, Email, Department, Address, IsActive	Hồ sơ thông tin liên hệ và đơn vị công tác của người mượn.
LoanSlips	LoanSlipId	SlipCode (UQ), BorrowerId (FK), CreatedBy (FK), BorrowDate, DueDate, Status	Đầu phiếu mượn thiết bị (Status: 0 = còn nợ, 1 = đã trả đủ).

Bảng	Khóa chính	Trường tiêu biểu	Ý nghĩa
LoanSlipDetails	DetailId	LoanSlipId (FK), DeviceId (FK), Quantity, ReturnedQuantity	Chi tiết từng thiết bị và tiến độ số lượng đã trả trong phiếu mượn.
ReturnRecords	ReturnRecordId	RecordCode (UQ), LoanSlipId (FK), ReturnDate, ReceivedBy (FK), GiverName, IsCancelled, CancelledBy, CancelReason	Đầu biên bản trả/bàn giao thiết bị và thông tin truy vết hủy biên bản.
ReturnRecordDetails	Id	ReturnRecordId (FK), DeviceId (FK), LoanSlipDetailId (FK), Quantity, ConditionCode, Condition	Chi tiết số lượng và tình trạng thiết bị trong từng lần trả (0..3).
CodeConfigs	ConfigId	EntityName (UQ), Prefix, UseDate, NumberLength, CurrentNumber, ResetDaily, LastResetDate	Quy tắc và bộ đếm sinh mã tự động cho từng đối tượng.

Hệ thống thiết lập các ràng buộc toàn vẹn chặt chẽ trong `03_Constraints.sql`: ràng buộc `CK_Devices_Inventory` bảo đảm `AvailableQuantity >= 0`, `DamagedQuantity >= 0`, `LostQuantity >= 0` và `AvailableQuantity + DamagedQuantity + LostQuantity <= TotalQuantity`; ràng buộc `CK_LoanSlipDetails_Returned` kiểm soát `0 <= ReturnedQuantity <= Quantity`; ràng buộc `CK_LoanSlips_Dates` kiểm tra `DueDate >= BorrowDate`; ràng buộc `CK_RRD_ConditionCode` giới hạn mã tình trạng trong tập `(0, 1, 2, 3)`; và ràng buộc `CK_CodeConfigs_Rules` bắt buộc `ResetDaily = 0 OR UseDate = 1` để ngăn việc đặt lại bộ đếm mỗi ngày khi mã không đính kèm ngày tháng. Bên cạnh đó, chỉ mục duy nhất `UQ_LoanSlipDetails_Slip_Device` ngăn một thiết bị xuất hiện lặp lại trên hai dòng của cùng một phiếu mượn, và 14 chỉ mục non-clustered trong `04_Indexes.sql` giúp tối ưu tốc độ tìm kiếm, lọc và nối bảng.

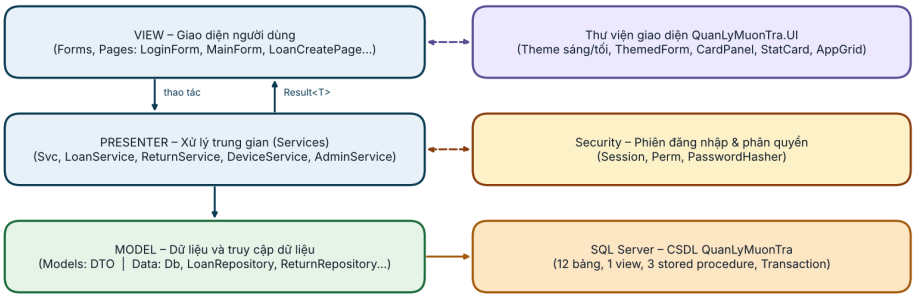
3.2.3. View, Stored procedure và transaction

Bảng 3.2. Các View và Stored Procedure trong cơ sở dữ liệu

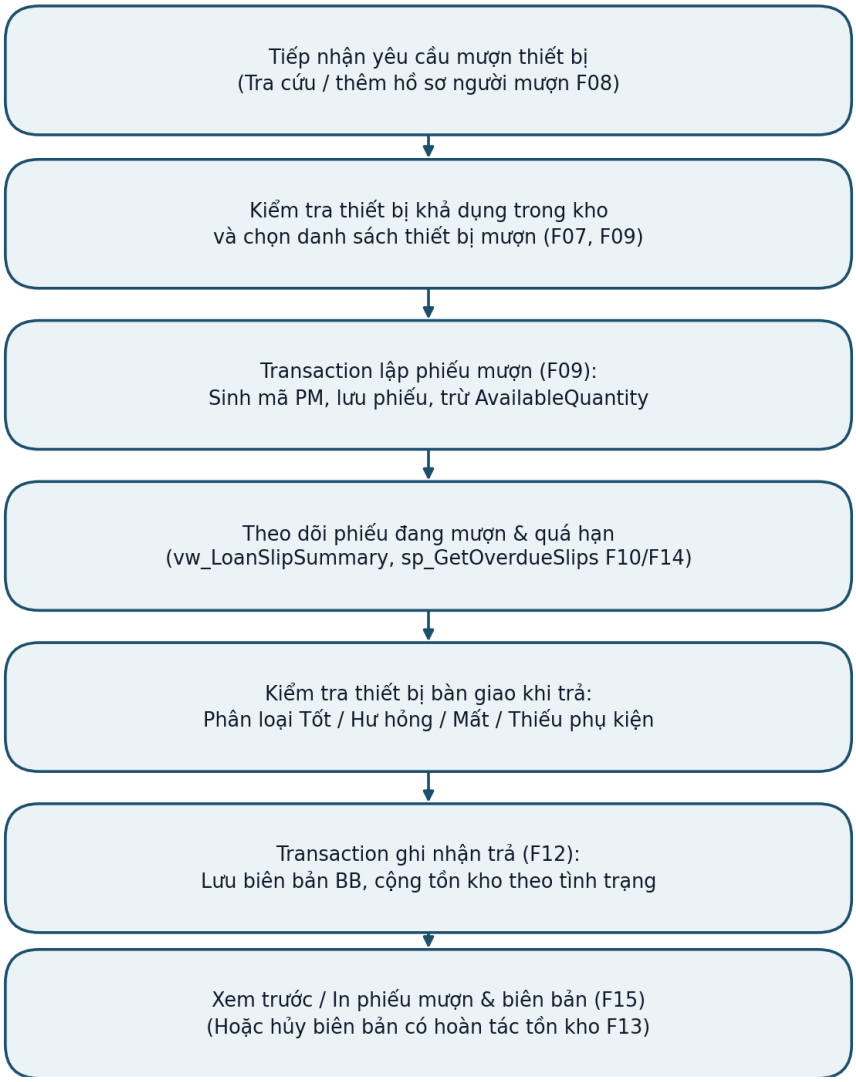
Đối tượng SQL	Loại	Mục đích và cơ chế xử lý
vw_LoanSlipSummary	View	Tổng hợp đầu phiếu mượn, thông tin người mượn, người lập, tổng số lượng mượn (TotalQty), đã trả (ReturnedQty), còn lại (RemainingQty), số ngày trễ (DaysOverdue) và tự động suy diễn trạng thái hiển thị DisplayStatus (0: Đang mượn, 1: Trả một phần, 2: Đã trả, 3: Quá hạn).
sp_GetNextCode	Stored Procedure	Sinh mã tự động theo bảng CodeConfigs. Sử dụng gợi ý khóa dòng WITH (UPDLOCK, HOLDLOCK) trong transaction để nhiều người dùng thao tác cùng lúc không bao giờ sinh trùng mã; hỗ trợ chế độ @Peek = 1 để xem trước mã mà không tăng bộ đếm.
sp_GetOverdueSlips	Stored Procedure	Truy vấn danh sách phiếu quá hạn thực tế theo ngày (@AsOfDate), hỗ trợ lọc từ khóa (@Keyword) và số ngày quá hạn tối thiểu (@MinDaysOverdue), đồng thời dùng CROSS APPLY + STRING_AGG để nối danh sách tên thiết bị và số lượng chưa trả.
sp_RecalculateInventory	Stored Procedure	Đồng bộ lại trạng thái phiếu mượn (LoanSlips.Status) và tính lại tồn kho khả dụng chuẩn xác theo công thức Available = Total - (Đang mượn) - Damaged - Lost; dùng sau khi nạp dữ liệu mẫu hoặc khi cần đối soát dữ liệu.

3.3. Kiến trúc và luồng xử lý

Ứng dụng được tổ chức thành 2 project trong cùng một solution: `QuanLyMuonTra.UI` (thư viện control và chủ đề giao diện) và `QuanLyMuonTra.App` (chứa các phân lớp `Forms`/`Pages`, `Services`, `Data`, `Models`, `Security`). Hình 3.2 minh họa kiến trúc MVP và mối liên hệ giữa các thành phần và Hình 3.3 thể hiện luồng xử lý nghiệp vụ mượn – trả thiết bị khép kín.



Hình 3.2. Mô hình kiến trúc MVP (Model – View – Presenter), SQL Server và thư viện UI



Hình 3.3. Quy trình nghiệp vụ từ lập phiếu mượn đến ghi nhận trả và in biên bản

3.4. Quy tắc nghiệp vụ và kiểm soát tồn kho

Khác với bài toán bán hàng thông thường, bài toán quản lý mượn trả thiết bị đòi hỏi bảo toàn tổng số lượng tài sản đầu tư trong suốt vòng đời cho mượn, trả lại, hư hỏng, sửa chữa hoặc thất lạc. Các công thức bất biến được duy trì xuyên suốt giữa tầng C# (`LoanRepository.cs`, `ReturnRepository.cs`) và tầng SQL Server (`03_Constraints.sql`, `05_StoredProcedures.sql`):

```
ĐangMượn (OnLoanQuantity) = Σ (Quantity − ReturnedQuantity) trên
↪ LoanSlipDetails
TổngSốLượng (TotalQuantity) = AvailableQuantity + OnLoanQuantity +
↪ DamagedQuantity + LostQuantity
CònPhảiTrả (RemainingQty) = TotalQty − ReturnedQty (trên từng dòng và
↪ toàn phiếu)
SốNgàyQuáHạn (DaysOverdue) = max(0, (HômNay − DueDate).Days) khi
↪ RemainingQty > 0 và DueDate < HômNay
```

Khi ghi nhận trả thiết bị tại `ReturnRepository.Create`, phương thức phân loại `Bucket(conditionCode, qty)` quy đổi số lượng trả vào 3 rổ tồn kho `(Good, Damaged, Lost)` theo đúng mã tình trạng: nếu trả ở tình trạng **Tốt (0)**, số lượng được cộng trả lại vào `AvailableQuantity` để sẵn sàng cho lượt mượn tiếp theo; nếu trả ở tình trạng **Hư hỏng (1)** hoặc **Thiếu phụ kiện (3)**, số lượng được cộng vào `DamagedQuantity` (chờ sửa chữa/bổ sung phụ kiện qua `RecoverDamaged`); nếu trả ở tình trạng **Mất (2)**, số lượng được cộng vào `LostQuantity`. Trạng thái hiển thị `DisplayStatus` của phiếu mượn không lưu cứng mà được tính động trên `vw_LoanSlipSummary`: ưu tiên kiểm tra nếu `RemainingQty = 0` là `Đã trả (2)`, ngược lại nếu `DueDate < HômNay` là `Quá hạn (3)`, nếu `ReturnedQty > 0` là `Trả một phần (1)`, và mặc định là `Đang mượn (0)`.

Bảng 3.3. Các tình huống kiểm soát nghiệp vụ

Tình huống nghiệp vụ	Cơ chế kiểm soát và kết quả kỳ vọng
Hai nhân viên cùng mượn một thiết bị sắp hết tồn	Câu lệnh UPDATE Devices ... WHERE AvailableQuantity >= @qty chỉ khớp cho giao dịch đến trước; giao dịch đến sau nhận rows = 0, ném BusinessException và rollback toàn bộ phiếu.

Tình huống nghiệp vụ	Cơ chế kiểm soát và kết quả kỳ vọng
Chọn trùng một thiết bị nhiều lần trên cùng phiếu mượn	Giao diện cộng dồn số lượng trong giỏ; lớp Data (thuộc Model) dùng GroupBy(i => i.DeviceId).OrderBy(g => g.Key) gộp thành 1 dòng duy nhất và khóa theo thứ tự ID cố định để tránh Deadlock.
Trả vượt quá số lượng còn phải trả của dòng phiếu	ReturnRepository khóa phiếu bằng WITH (UPDLOCK, HOLDLOCK), đối chiếu tổng số lượng trả với (Quantity - ReturnedQuantity); nếu vượt sẽ từ chối và rollback.
Trả thiết bị hư hỏng hoặc thiếu phụ kiện nhưng bỏ trống ghi chú	ReturnService.Validate bắt buộc nhập ConditionNote chi tiết khi Condition là Damaged hoặc MissingAccessory; từ chối lưu nếu để trống.
Hủy biên bản trả sau khi thiết bị vừa trả đã được cho mượn lại	ReturnRepository.Cancel kiểm tra AvailableQuantity >= @good AND DamagedQuantity >= @damaged AND LostQuantity >= @lost; nếu tồn kho hiện tại không đủ để trừ hoàn tác thì từ chối hủy.
Bật đặt lại bộ đếm mã hằng ngày nhưng không chèn ngày vào mã	AdminService.SaveCodeConfig và CK_CodeConfigs_Rules chặn cấu hình (ResetDaily = 1 khi UseDate = 0) nhằm ngăn sinh trùng mã vào ngày hôm sau.

3.5. Ma trận phân quyền người dùng

Trong kịch bản khởi tạo `06_SeedData.sql`, hệ thống định nghĩa sẵn 17 mã quyền chức năng và 2 vai trò mẫu: **Người quản lý** (được gán toàn bộ 17 quyền) và **Nhân viên kho** (được gán 10 quyền phục vụ công việc vận hành kho hằng ngày). Người quản lý có thể thay đổi tập quyền này bất kỳ lúc nào trên màn hình F05.

Bảng 3.4. Ma trận phân quyền mặc định theo hai vai trò

Mã quyền (Permission Code)	Tên quyền chức năng	Nhân viên kho	Người quản lý
DASHBOARD_VIEW	Xem màn hình Tổng quan (Dashboard F02)	Có	Có
CATEGORY_VIEW	Xem danh mục loại thiết bị (F06)	Có	Có

Mã quyền (Permission Code)	Tên quyền chức năng	Nhân viên kho	Người quản lý
CATEGORY_MANAGE	Thêm, sửa, xóa loại thiết bị (F06)	Không	Có
DEVICE_MANAGE	Quản lý thiết bị và phục hồi hư hỏng (F07)	Có	Có
BORROWER_MANAGE	Quản lý hồ sơ người mượn và lịch sử (F08)	Có	Có
LOAN_CREATE	Lập phiếu mượn thiết bị (F09)	Có	Có
LOAN_VIEW	Tra cứu và xem chi tiết phiếu mượn (F10, F11)	Có	Có
RETURN_CREATE	Ghi nhận trả thiết bị (F12)	Có	Có
RECORD_VIEW	Tra cứu và xem chi tiết biên bản trả (F13)	Có	Có
RECORD_CANCEL	Hủy biên bản trả / bàn giao (F13)	Không	Có
OVERDUE_VIEW	Xem danh sách phiếu quá hạn (F14)	Có	Có
REPORT_PRINT	Xem trước và in phiếu mượn, biên bản trả (F15)	Có	Có
REPORT_VIEW	Xem số liệu báo cáo tổng hợp mượn trả & tồn kho (F15)	Không	Có
STATS_VIEW	Xem thống kê thiết bị được mượn nhiều (F18)	Không	Có
USER_MANAGE	Quản lý tài khoản, khóa/mở khóa, đặt lại mật khẩu (F04)	Không	Có
ROLE_MANAGE	Quản lý nhóm quyền và gán quyền chức năng (F05)	Không	Có
CODE_CONFIG	Cấu hình quy tắc sinh mã tự động (F16)	Không	Có

CHƯƠNG 4

TRIỂN KHAI VÀ KẾT QUẢ

4.1. Môi trường phát triển

Bảng 4.1. Công nghệ và môi trường triển khai

Thành phần	Công nghệ hoặc yêu cầu
Hệ điều hành	Windows 10 / Windows 11 (64-bit)
Ngôn ngữ & Nền tảng	C# trên .NET 10 (target framework: net10.0-windows)
Công cụ phát triển (IDE)	Visual Studio 2022 / 2026 có workload .NET desktop development
Giao diện Desktop (UI)	Windows Forms + Thư viện tự xây dựng QuanLyMuonTra.UI (GDI+, Segoe MDL2 Assets)
Hệ quản trị CSDL	Microsoft SQL Server 2017 trở lên (Express / Developer / LocalDB)
Truy cập dữ liệu (Model/Data)	Dapper 2.x kết hợp Microsoft.Data.SqlClient 6.x (ADO.NET)
Bảo mật mật khẩu	PBKDF2-SHA256 (100.000 vòng lặp, Salt 16 byte, Key 32 byte)
In ấn biểu mẫu	System.Drawing.Printing (PrintDocument & PrintPreviewDialog)
Quản lý cấu hình	Tệp văn bản connection.txt đặt cạnh tệp thực thi (.exe)

4.2. Cấu trúc Project

4.2.1. Sơ đồ tổng quan cấu trúc thư mục

```
QuanLyMuonTra.slnx
|-- QuanLyMuonTra.UI/         (Thư viện control & chủ đề Sáng/Tối)
|   |-- Theme.cs, ThemedBase.cs, Surfaces.cs
|   `-- Inputs.cs, Widgets.cs, Gfx.cs
|-- QuanLyMuonTra.App/        (Ứng dụng chính Windows Forms)
|   |-- Forms/                (LoginForm, MainForm, PrintHelper & 10
↪  Dialog)
|   |-- Pages/                (14 UserControl chức năng F02, F04-F16, F18)
|   |-- Services/             (Presenter: nghiệp vụ & kiểm tra quyền)
|   |-- Data/                 (Model: truy vấn SQL Server với Dapper)
|   |-- Models/               (DTO và các lớp bộ lọc truy vấn)
|   |-- Security/             (Session.cs, Perm.cs, PasswordHasher.cs)
|   |-- NavCatalog.cs, Program.cs, connection.txt
```

```
|-- Database/                  (01_CreateDatabase.sql -> 07_TestData.sql)
`-- docs/                      (TEST_CASES.md, BAO_CAO_NHOM_TEMPLATE.md...)
```

4.2.2. Vai trò của các thành phần

`Program.cs` khởi tạo cấu hình ứng dụng và chạy vòng lặp xác thực: mở `Login Form` để đăng nhập, nếu thành công thì khởi chạy `MainForm`; nếu người dùng nhấn `Đăng xuất` (`main.LoggedOut == true`), chương trình xóa phiên `Session.End()` và quay lại `LoginForm` mà không cần tắt ứng dụng. Các lớp trong `Pages/` đảm nhiệm từng màn hình nghiệp vụ lớn được nhúng vào `MainForm`, trong khi các hộp thoại trong `Forms/` (`Device EditDialog`, `BorrowerEditDialog`, `LoanSlipDetailDialog`, `ReturnCreate Dialog`...) xử lý tác vụ nhập liệu hoặc xem chi tiết chuyên sâu.

4.3. Cấu hình và chạy chương trình

Bước 1: Cài đặt .NET 10 SDK, Visual Studio (có workload .NET desktop development) và Microsoft SQL Server (2017 trở lên). Bước 2: Mở SQL Server Management Studio (SSMS), kết nối tới SQL Server và chạy lần lượt các tập lệnh trong thư mục `Database/` theo đúng thứ tự từ `01_CreateDatabase.sql` đến `06_SeedData.sql` (và `07_TestData.sql` nếu muốn nạp 5 phiếu mượn và 3 biên bản kiểm thử). Bước 3: Mở tệp `QuanLyMuonTra.App/connection.txt` và cập nhật chuỗi kết nối phù hợp với máy tính triển khai:

```
Server=.\SQLEXPRESS;Database=QuanLyMuonTra;Trusted_Connection=True;Trust
↪  ServerCertificate=True;
```

Bước 4: Mở solution `QuanLyMuonTra.slnx` bằng Visual Studio hoặc sử dụng dòng lệnh .NET CLI để khôi phục gói NuGet, biên dịch và chạy ứng dụng:

```
dotnet restore .\QuanLyMuonTra.slnx
dotnet build .\QuanLyMuonTra.slnx
dotnet run --project .\QuanLyMuonTra.App\QuanLyMuonTra.App.csproj
```

Bước 5: Đăng nhập thử nghiệm bằng hai tài khoản mẫu được tạo sẵn trong `06_Seed Data.sql`: tài khoản Người quản lý (`admin` / `Admin@123`) hoặc tài khoản Nhân viên kho (`kho01` / `Kho@123`).

4.4. Chi tiết các chức năng quan trọng

4.4.1. Kiểm soát truy cập, băm mật khẩu và phân quyền động

Quá trình xác thực tại `UserRepository.Authenticate` truy vấn thông tin người dùng theo `Username`, kiểm tra mật khẩu thông qua `PasswordHasher.Verify` (PBKDF2-SHA256 100.000 vòng lặp) và kiểm tra trạng thái `IsActive`. Nếu hợp lệ, danh sách mã quyền của vai trò tương ứng được nạp vào `Session`. Mọi phương thức ở Presenter (`Services`) đều đi qua hàm bọc `Svc.Run(permissions, work)`: nếu người dùng chưa đăng nhập hoặc không sở hữu ít nhất một quyền trong mảng yêu cầu, hàm lập tức trả về `Result.Fail("Bạn không có quyền thực hiện chức năng này.")` trước khi chạm tới cơ sở dữ liệu.

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Màn hình đăng nhập (LoginForm) khi nhập tài khoản admin/kho01 hoặc khi thông báo lỗi sai mật khẩu / tài khoản bị khóa

(Thành phần mã nguồn: QuanLyMuonTra.App/Forms/LoginForm.cs)

Hình 4.1. Giao diện đăng nhập và thông báo kiểm tra tài khoản/mật khẩu

4.4.2. Lập phiếu mượn nhiều thiết bị và trừ tồn kho có điều kiện

Tại `LoanCreatePage`, nhân viên có thể tìm kiếm nhanh các thiết bị còn tồn kho, chọn số lượng và thêm nhiều thiết bị vào giỏ mượn. Khi lưu phiếu, `LoanService.Validate` kiểm tra người mượn hợp lệ, ngày mượn không ở tương lai (`BorrowDate <= Today`), hạn trả không trước ngày mượn (`DueDate >= BorrowDate`) và số lượng mỗi dòng lớn hơn 0. Tiếp đó, `LoanRepository.Create` mở transaction qua `Db.InTransaction`, sinh mã phiếu mới bằng `CodeRepository.Next`, tạo bản ghi `LoanSlips` và duyệt qua từng thiết bị theo thứ tự `DeviceId` tăng dần. Câu lệnh `UPDATE Devices SET AvailableQuantity = AvailableQuantity - @qty WHERE DeviceId = @deviceId AND IsActive = 1 AND AvailableQuantity >= @qty` bảo đảm việc kiểm tra và trừ tồn diễn ra nguyên tử (atomic); nếu `rows == 0`, toàn bộ giao dịch lập tức rollback.

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Màn hình Lập phiếu mượn (LoanCreatePage) với danh sách thiết bị khả dụng bên trái và giỏ thiết bị chọn mượn bên phải

(Thành phần mã nguồn: QuanLyMuonTra.App/Pages/LoanCreatePage.cs)

Hình 4.2. Giao diện lập phiếu mượn nhiều thiết bị và giỏ thiết bị (F09)

4.4.3. Ghi nhận trả theo tình trạng và hoàn tác khi hủy biên bản

Nghiệp vụ trả thiết bị (`ReturnCreateDialog` và `ReturnRepository.Create`) cho phép người mượn trả từng phần hoặc trả toàn bộ thiết bị trên phiếu, đồng thời cho phép tách một thiết bị thành các dòng trạng thái khác nhau (Tốt, Hư hỏng, Mất, Thiếu phụ kiện). Trong transaction, hệ thống khóa dòng `LoanSlips` và `LoanSlipDetails` bằng `WITH (UPDLOCK, HOLDLOCK)`, kiểm tra tổng số lượng trả của từng mục không vượt số lượng còn nợ (`Quantity - ReturnedQuantity`), tạo biên bản `ReturnRecords` + `ReturnRecordDetails`, tăng `ReturnedQuantity` và phân bổ số lượng vào đúng cột `AvailableQuantity`, `DamagedQuantity` hoặc `LostQuantity`. Cuối cùng, câu lệnh `RecomputeStatusSql` tự động chuyển `LoanSlips.Status = 1` nếu toàn bộ các dòng trong phiếu đã được trả đủ. Khi cần hủy một biên bản lập nhầm (`Return Repository.Cancel`), hệ thống kiểm tra tồn kho hiện tại có đủ để trừ ngược hay không trước khi hoàn tác và đánh dấu `IsCancelled = 1`.

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Hộp thoại Ghi nhận trả (ReturnCreateDialog) hiển thị số lượng còn phải trả, cột chọn Tình trạng (Tốt/Hư hỏng/Mất/Thiếu phụ kiện) và ghi chú

(Thành phần mã nguồn: QuanLyMuonTra.App/Forms/ReturnCreateDialog.cs)

Hình 4.3. Hộp thoại ghi nhận trả thiết bị theo tình trạng (F12)

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Cửa sổ PrintPreviewDialog hiển thị mẫu in Phiếu mượn thiết bị hoặc Biên bản trả / bàn giao thiết bị kèm phần chữ ký

(Thành phần mã nguồn: QuanLyMuonTra.App/Forms/PrintHelper.cs)

Hình 4.4. Cửa sổ xem trước bản in phiếu mượn và biên bản bàn giao (F15)

4.4.4. Báo cáo tổng hợp, thống kê thiết bị mượn nhiều và in ấn

Chức năng báo cáo (`ReportPage` và `AdminRepository.Summary`) tổng hợp đồng thời các chỉ số giao dịch trong kỳ lọc (`SlipCount`, `BorrowedQuantity`, `ReturnedQuantity`, `OutstandingQuantity`, `OverdueSlipCount`) và các chỉ số tồn kho hiện hành (`ActiveDeviceCount`, `AvailableQuantity`, `DamagedQuantity`, `LostQuantity`), kết hợp hiển thị bảng xếp hạng Top thiết bị được mượn nhiều nhất. Đối với chức năng thống kê chuyên sâu (`PopularDevicePage` – F18), người dùng có thể tùy chỉnh khoảng ngày và giới hạn `Top N` (từ 1 đến 100) để phân tích nhu cầu sử dụng thiết bị thực tế.

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Màn hình Báo cáo & in ấn (ReportPage) hoặc Thiết bị mượn nhiều (PopularDevicePage) với các chỉ số tổng hợp dưới chân trang

(Thành phần mã nguồn: QuanLyMuonTra.App/Pages/ReportPage.cs, PopularDevicePage.cs)

Hình 4.5. Màn hình báo cáo tổng hợp và thống kê thiết bị mượn nhiều (F15, F18)

4.4.5. Sinh mã tự động an toàn đồng thời và chế độ sáng/tối

Màn hình `CodeConfigPage` (F16) quản lý bảng `CodeConfigs` và kết nối với thủ tục `dbo.sp_GetNextCode`. Khi người dùng nhấn `Xem mã tiếp theo`, hệ thống gọi `CodeRepository.Peek` (`@Peek = 1`) để hiển thị chuỗi mã dự kiến (ví dụ `PM-20261107-0001` hoặc `TB0009`) mà không làm tăng bộ đếm trong cơ sở dữ liệu. Ngoài ra, khi người dùng

tự nhập tay mã thiết bị dạng `TB0012`, hàm `CodeRepository.SyncCounter` tự động kéo `CurrentNumber` lên theo số lớn nhất để các lần gợi ý sau không bị trùng. Về trải nghiệm thị giác, lớp `Theme` và `ThemeApplier` cho phép chuyển đổi tức thời giữa giao diện Sáng (`#F5F7FA` / `#FFFFFF`) và giao diện Tối (`#0F172A` / `#1E293B`) ngay trên thanh tiêu đề.

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Màn hình Cấu hình mã tự động (CodeConfigPage) với lưới 4 đối tượng (LoanSlip, ReturnRecord, Device, Borrower) và thanh chỉnh sửa phía dưới

(Thành phần mã nguồn: QuanLyMuonTra.App/Pages/CodeConfigPage.cs)

Hình 4.6. Màn hình cấu hình quy tắc sinh mã tự động và xem trước mã (F16)

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Giao diện MainForm và nội dung trang khi kích hoạt chế độ Giao diện tối (Dark Mode)

(Thành phần mã nguồn: QuanLyMuonTra.UI/Theme.cs, QuanLyMuonTra.App/Forms/MainForm.cs)

Hình 4.7. Giao diện chính khi chuyển đổi chế độ Sáng / Tối (ThemeMode)

4.5. Hình ảnh sản phẩm thực tế

Các hình dưới đây là ảnh chụp màn hình thực tế khi chạy ứng dụng trên Windows, được sắp xếp theo thứ tự chức năng của hệ thống.

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Màn hình Dashboard khi đăng nhập bằng tài khoản kho01 (chỉ hiển thị các menu được phân quyền cho nhân viên kho)

(Thành phần mã nguồn: QuanLyMuonTra.App/Forms/MainForm.cs, Pages/DashboardPage.cs)

Hình 4.8. Màn hình Tổng quan (Dashboard F02) của tài khoản Nhân viên kho

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Màn hình Dashboard khi đăng nhập bằng tài khoản admin (hiển thị đầy đủ 4 thẻ thống kê, danh sách phiếu gần đây và toàn bộ menu quản trị)

(Thành phần mã nguồn: QuanLyMuonTra.App/Forms/MainForm.cs, Pages/DashboardPage.cs)

Hình 4.9. Màn hình Tổng quan (Dashboard F02) của tài khoản Người quản lý

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Màn hình Quản lý thiết bị (DevicePage) và hộp thoại Thêm/Sửa thiết bị (DeviceEditDialog)

(Thành phần mã nguồn: QuanLyMuonTra.App/Pages/DevicePage.cs, Forms/DeviceEditDialog.cs)

Hình 4.10. Giao diện quản lý danh mục thiết bị và lọc tồn kho (F07)

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Màn hình Loại thiết bị (CategoryPage) và hộp thoại Thêm/Cập nhật loại thiết bị (CategoryEditDialog)

(Thành phần mã nguồn: QuanLyMuonTra.App/Pages/CategoryPage.cs, Forms/CategoryEditDialog.cs)

Hình 4.11. Giao diện quản lý loại thiết bị (F06)

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Màn hình Người mượn (BorrowerPage), hộp thoại BorrowerEditDialog và cửa sổ Lịch sử mượn (BorrowerHistoryDialog)

(Thành phần mã nguồn: QuanLyMuonTra.App/Pages/BorrowerPage.cs, Forms/BorrowerHistoryDialog.cs)

Hình 4.12. Giao diện quản lý hồ sơ người mượn và lịch sử mượn (F08)

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Màn hình Tra cứu phiếu mượn (LoanSearchPage) có badge trạng thái và hộp thoại Chi tiết phiếu mượn (LoanSlipDetailDialog)

(Thành phần mã nguồn: QuanLyMuonTra.App/Pages/LoanSearchPage.cs, Forms/LoanSlipDetailDialog.cs)

Hình 4.13. Giao diện tra cứu phiếu mượn và hộp thoại chi tiết phiếu (F10, F11)

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Màn hình Biên bản trả / bàn giao (ReturnRecordPage) và màn hình Phiếu quá hạn (OverduePage)

(Thành phần mã nguồn: QuanLyMuonTra.App/Pages/ReturnRecordPage.cs, OverduePage.cs)

Hình 4.14. Giao diện quản lý biên bản trả và danh sách phiếu quá hạn (F13, F14)

[CHỪA CHỖ CHÈN ẢNH CHỤP MÀN HÌNH THỰC TẾ]

Nội dung cần chụp: Màn hình Người dùng (UserPage) và màn hình Nhóm quyền (RolePage) với danh sách 17 quyền chức năng

(Thành phần mã nguồn: QuanLyMuonTra.App/Pages/UserPage.cs, RolePage.cs)

Hình 4.15. Giao diện quản lý người dùng và phân quyền nhóm quyền (F04, F05)

4.6. Kiểm thử và đánh giá kết quả

Nhóm đã xây dựng tệp `Database/07_TestData.sql` (tạo sẵn 5 phiếu mượn từ `PM-0001` đến `PM-0005` và 3 biên bản trả từ `BB-0001` đến `BB-0003`, bao phủ các trạng thái: quá hạn chưa trả, đang mượn trong hạn, đã trả đủ, trả một phần và trả thiết bị hư hỏng) cùng tài liệu `docs/TEST_CASES.md` gồm 35 ca kiểm thử chức năng và kịch bản kiểm thử đồng thời. Việc kiểm thử được thực hiện thủ công trên máy Windows với SQL Server Express và .NET 10; kết quả từng ca được ghi trong Bảng 4.2.

Bảng 4.2. Kết quả thực thi 35 kịch bản kiểm thử

Mã TC	Chức năng	Dữ liệu / Thao tác kiểm thử	Kết quả kỳ vọng	Kết quả thực tế	Trạng thái
TC01	Đăng nhập	Đăng nhập admin với mật khẩu mẫu Admin@123	Vào Dashboard với toàn quyền quản lý	Đúng kỳ vọng	Đạt

Mã TC	Chức năng	Dữ liệu / Thao tác kiểm thử	Kết quả kỳ vọng	Kết quả thực tế	Trạng thái
TC02	Đăng nhập	Nhập sai mật khẩu	Báo thông tin đăng nhập không hợp lệ	Đúng kỳ vọng	Đạt
TC03	Đăng nhập	Đăng nhập tài khoản bị khóa (IsActive = 0)	Từ chối đăng nhập và báo tài khoản bị khóa	Đúng kỳ vọng	Đạt
TC04	Phân quyền	Đăng nhập tài khoản kho01 (Kho@123)	Chỉ hiển thị các chức năng được cấp quyền	Đúng kỳ vọng	Đạt
TC05	F06 - Loại TB	Tạo loại thiết bị với mã mới hợp lệ	Lưu thành công và hiển thị trên lưới	Đúng kỳ vọng	Đạt
TC06	F06 - Loại TB	Tạo loại thiết bị trùng mã hoặc tên đang dùng	Báo trùng, không tạo bản ghi mới	Đúng kỳ vọng	Đạt
TC07	F07 - Thiết bị	Thêm thiết bị mới với số lượng hợp lệ	Lưu thành công, khả dụng bằng tổng số lượng	Đúng kỳ vọng	Đạt
TC08	F07 - Thiết bị	Sửa giảm tổng số lượng nhỏ hơn số đang mượn + hỏng/mất	Từ chối dữ liệu không hợp lệ	Đúng kỳ vọng	Đạt
TC09	F08 - Người mượn	Tạo người mượn mới với thông tin hợp lệ	Lưu thành công và cấp mã tự động NMxxxx	Đúng kỳ vọng	Đạt
TC10	F08 - Người mượn	Xóa người mượn đã có lịch sử phiếu mượn	Từ chối xóa, hướng dẫn chuyển ngừng hoạt động	Đúng kỳ vọng	Đạt
TC11	F09 - Lập phiếu	Lập phiếu mượn với 1 thiết bị đủ tồn khả dụng	Tạo phiếu thành công, giảm AvailableQuantity đúng SL	Đúng kỳ vọng	Đạt
TC12	F09 - Lập phiếu	Mượn số lượng lớn hơn số lượng khả dụng	Báo thiếu tồn kho; phiếu không được lưu	Đúng kỳ vọng	Đạt
TC13	F09 - Lập phiếu	Lập phiếu mượn gồm nhiều thiết bị khác nhau	Tạo 1 phiếu với đầy đủ các dòng chi tiết	Đúng kỳ vọng	Đạt
TC14	F10 / F11	Tìm phiếu theo mã và mở hộp thoại chi tiết	Hiển thị đúng đầu phiếu, thiết bị và lịch sử trả	Đúng kỳ vọng	Đạt
TC15	F12 - Trả TB	Trả một phần thiết bị ở tình trạng Tốt	Tăng ReturnedQuantity và AvailableQuantity đúng SL	Đúng kỳ vọng	Đạt

Mã TC	Chức năng	Dữ liệu / Thao tác kiểm thử	Kết quả kỳ vọng	Kết quả thực tế	Trạng thái
TC16	F12 - Trả TB	Trả nốt phần còn lại ở tình trạng Tốt	Phiếu chuyển sang Đã trả; tồn kho khả dụng tăng đúng	Đúng kỳ vọng	Đạt
TC17	F12 - Trả TB	Trả thiết bị ở tình trạng Hư hỏng (kèm ghi chú)	Tăng DamagedQuantity, không tăng AvailableQuantity	Đúng kỳ vọng	Đạt
TC18	F12 - Trả TB	Trả thiết bị ở tình trạng Mất	Tăng LostQuantity, không tăng AvailableQuantity	Đúng kỳ vọng	Đạt
TC19	F12 - Trả TB	Trả tình trạng Hư hỏng / Thiếu phụ kiện nhưng trống ghi chú	Từ chối lưu và yêu cầu nhập ghi chú tình trạng	Đúng kỳ vọng	Đạt
TC20	F12 - Trả TB	Nhập số lượng trả vượt số lượng còn phải trả	Từ chối lưu; dữ liệu transaction không thay đổi	Đúng kỳ vọng	Đạt
TC21	F13 - Biên bản	Tra cứu biên bản trả theo mã biên bản / mã phiếu	Hiển thị đúng biên bản bàn giao tương ứng	Đúng kỳ vọng	Đạt
TC22	F13 - Biên bản	Hủy biên bản với lý do hợp lệ khi tồn kho cho phép	Biên bản chuyển Đã hủy, hoàn tác tồn kho và phiếu mượn	Đúng kỳ vọng	Đạt
TC23	F13 - Biên bản	Hủy biên bản sau khi thiết bị vừa trả đã được mượn lại	Từ chối hủy do không thể hoàn tác tồn kho an toàn	Đúng kỳ vọng	Đạt
TC24	F14 - Quá hạn	Mở danh sách phiếu quá hạn	Chỉ hiện các phiếu đã qua DueDate và còn thiết bị nợ	Đúng kỳ vọng	Đạt
TC25	F14 - Quá hạn	Lọc theo số ngày quá hạn tối thiểu	Chỉ hiển thị các phiếu có DaysOverdue >= mức lọc	Đúng kỳ vọng	Đạt
TC26	F04 - Tài khoản	Tạo tài khoản người dùng mới	Lưu thành công, mật khẩu được băm PBKDF2-SHA256	Đúng kỳ vọng	Đạt
TC27	F04 - Tài khoản	Đặt lại mật khẩu cho tài khoản	Mật khẩu mới đăng nhập được, mật khẩu cũ hết hiệu lực	Đúng kỳ vọng	Đạt
TC28	F04 - Tài khoản	Thử tự khóa chính tài khoản đang đăng nhập	Hệ thống từ chối thao tác tự khóa	Đúng kỳ vọng	Đạt
TC29	F05 - Nhóm quyền	Thay đổi quyền của nhóm Nhân viên kho và lưu	Quyền lưu trong transaction và áp dụng ở lần đăng nhập kế	Đúng kỳ vọng	Đạt
TC30	F16 - Mã tự động	Đổi tiền tố mã và nhấn Xem mã tiếp theo	Mã xem trước sử dụng tiền tố mới, không tăng bộ đếm	Đúng kỳ vọng	Đạt
TC31	F16 - Mã tự động	Bật Reset mỗi ngày nhưng tắt Chèn ngày vào mã	Từ chối lưu cấu hình để ngăn trùng mã	Đúng kỳ vọng	Đạt

Mã TC	Chức năng	Dữ liệu / Thao tác kiểm thử	Kết quả kỳ vọng	Kết quả thực tế	Trạng thái
TC32	F15 - Báo cáo	Xem báo cáo tổng hợp theo khoảng ngày	Số liệu phiếu và tồn kho khớp với dữ liệu thực tế	Đúng kỳ vọng	Đạt
TC33	F15 - In ấn	Xem trước / in Phiếu mượn thiết bị	Bản in hiển thị đầy đủ đầu phiếu, chi tiết và mục ký tên	Đúng kỳ vọng	Đạt
TC34	F15 - In ấn	Xem trước / in Biên bản trả thiết bị	Bản in hiển thị đúng số lượng, tình trạng và mục ký tên	Đúng kỳ vọng	Đạt
TC35	F18 - Thống kê	Xem bảng xếp hạng thiết bị mượn nhiều	Sắp xếp đúng theo tổng số lượng mượn giảm dần	Đúng kỳ vọng	Đạt

Kịch bản kiểm thử đồng thời (Concurrency Test): mở hai phiên ứng dụng bằng hai tài khoản nhân viên kho và cùng lập phiếu mượn một thiết bị khi số lượng khả dụng chỉ còn đủ cho một phiếu. Kết quả kỳ vọng: chỉ một giao dịch thành công, giao dịch còn lại nhận thông báo thiếu tồn kho, tồn kho không bị âm và không sinh phiếu mượn dở dang. Kết quả này được bảo đảm bởi câu lệnh `UPDATE` có điều kiện `AvailableQuantity >= @qty` trong cùng một transaction (nếu không cập nhật được dòng nào thì toàn bộ giao dịch rollback), cùng cơ chế khóa dòng theo thứ tự `DeviceId` cố định và tự thử lại tối đa 3 lần khi gặp deadlock (mã lỗi 1205) tại `Db.InTransaction`. Kết quả thực hiện: đạt, đúng như kỳ vọng.

Môi trường kiểm thử. Hệ điều hành: Windows; hệ quản trị cơ sở dữ liệu: SQL Server Express; công cụ: Visual Studio với .NET 10; người thực hiện kiểm thử: Ngô Minh Hải; ngày kiểm thử: 08/11/2026.

Đánh giá. Cả 35 ca kiểm thử đều cho kết quả đúng với kỳ vọng (35/35 ca đạt), bao gồm các ca đăng nhập và phân quyền, quản lý danh mục, lập phiếu mượn, ghi nhận trả theo tình trạng, hủy biên bản, phiếu quá hạn, quản trị hệ thống, cấu hình mã tự động, báo cáo và in ấn. Các ràng buộc nghiệp vụ quan trọng (không mượn vượt tồn kho, không trả vượt số còn nợ, bắt buộc ghi chú khi thiết bị hư hỏng hoặc thiếu phụ kiện) đều được hệ thống từ chối đúng cách mà không làm thay đổi dữ liệu.

CHƯƠNG 5

TỔNG KẾT

5.1. Kết quả đạt được

Đối chiếu với mã nguồn trên repository GitHub của nhóm, nhóm đã phân tích, thiết kế và xây dựng hoàn chỉnh ứng dụng desktop **Quản lý mượn trả thiết bị** trên nền tảng C# Windows Forms (.NET 10) và SQL Server. Hệ thống đáp ứng đầy đủ 16 chức năng (F01–F02, F04–F16, F18) từ quản lý danh mục (loại thiết bị, thiết bị, người mượn), xử lý nghiệp vụ cốt lõi (lập phiếu mượn nhiều thiết bị, tra cứu, xem chi tiết, ghi nhận trả theo 4 tình trạng, quản lý và hủy biên bản bàn giao, theo dõi phiếu quá hạn) đến quản trị hệ thống (người dùng, nhóm quyền động RBAC, cấu hình mã tự động) và báo cáo – thống kê – in ấn.

Về mặt kỹ thuật, cơ sở dữ liệu gồm 12 bảng, 1 view và 3 stored procedure được thiết kế có khả năng chạy lặp lại an toàn (idempotent scripts), kết hợp chặt chẽ giữa ràng buộc `CHECK/UNIQUE` ở tầng SQL và `SqlTransaction` có cơ chế tự động thử lại khi gặp Deadlock ở lớp Data (Model). Bộ thư viện giao diện riêng `QuanLyMuonTra.UI` mang lại phong cách hiện đại, hỗ trợ chuyển đổi chủ đề Sáng/Tối mượt mà và không phụ thuộc thư viện giao diện thương mại.

5.2. Hạn chế

- Truy cập dữ liệu hiện sử dụng Dapper và câu lệnh SQL tham số hóa; chưa tích hợp Entity Framework Core (`DbContext`/`LINQ to Entities`) nếu học phần có yêu cầu chấm điểm riêng cho ORM này.
- Biểu mẫu in ấn đang được vẽ trực tiếp bằng GDI+ (`PrintDocument` trong `PrintHelper.cs`) và thống kê hiển thị dạng bảng số liệu; chưa tích hợp công cụ xuất tệp Excel (`.xlsx`), PDF trực tiếp hoặc biểu đồ trực quan trên trang báo cáo.
- Các cột chữ ký số (`BorrowerSignature`, `ReceiverSignature`) đã được thiết kế sẵn trong bảng `LoanSlips` và `ReturnRecords` nhưng trên giao diện hiện tại mới dừng ở bước in khung ký tên trên giấy.
- Chưa có bộ cài đặt tự động (Installer/MSI) và hộp thoại cấu hình chuỗi kết nối SQL Server trực tiếp trên giao diện (hiện chỉnh sửa qua tệp `connection.txt`).
- Dự án chưa có bộ kiểm thử tự động (unit test/integration test); việc kiểm thử hiện thực

hiện thủ công theo 35 ca trong `docs/TEST_CASES.md` trên máy Windows có .NET 10 và SQL Server, kết quả chi tiết ghi tại Bảng 4.2.

5.3. Hướng phát triển

Trong giai đoạn tiếp theo, hệ thống có thể được nâng cấp theo các hướng thiết thực: (1) Bổ sung chức năng xuất báo cáo tổng hợp và danh sách phiếu quá hạn ra tệp Excel (`.xlsx`) bằng thư viện ClosedXML hoặc EPPlus; (2) Vẽ biểu đồ trực quan thể hiện xu hướng mượn trả theo tháng và tỷ lệ thiết bị hư hỏng/mất ngay trên Dashboard và trang Báo cáo; (3) Tích hợp quét mã vạch (Barcode / QR Code) bằng đầu đọc USB hoặc camera để chọn nhanh thiết bị và người mượn khi lập phiếu; (4) Xây dựng hộp thoại kiểm tra và lưu cấu hình kết nối SQL Server ngay từ màn hình đăng nhập; và (5) Mở rộng cổng thông tin web hoặc dịch vụ gửi email tự động nhắc người mượn khi phiếu sắp đến hạn hoặc quá hạn trả.

5.4. Bài học kinh nghiệm

Quá trình phát triển đồ án giúp nhóm rút ra nhiều bài học thực tiễn: trong các hệ thống quản lý kho và tài sản, việc kiểm tra điều kiện trên giao diện là chưa đủ mà bắt buộc phải khóa bản ghi và kiểm tra điều kiện ngay trong câu lệnh `UPDATE` bên trong Transaction để chống xung đột đồng thời. Bên cạnh đó, việc tách bạch thư viện `QuanLyMuonTra.UI` và lớp bọc nghiệp vụ `Svc.Run` giúp mã nguồn các trang giao diện ngắn gọn, dễ đọc, đồng nhất cách xử lý lỗi tiếng Việt và dễ dàng bảo trì qua nhiều giai đoạn.

TÀI LIỆU THAM KHẢO

TÀI LIỆU THAM KHẢO

[1] Microsoft Learn, "Windows Forms Overview," https://learn.microsoft.com/en-us/dotnet/desktop/winforms/overview/ (truy cập ngày 06/11/2026).

[2] Microsoft Learn, "C# documentation," https://learn.microsoft.com/en-us/dotnet/csharp/ (truy cập ngày 06/11/2026).

[3] Microsoft Learn, "Language Integrated Query (LINQ)," https://learn.microsoft.com/en-us/dotnet/csharp/linq/ (truy cập ngày 06/11/2026).

[4] Microsoft Learn, "Microsoft.Data.SqlClient Overview," https://learn.microsoft.com/en-us/sql/connect/ado-net/overview-sqlclient-driver (truy cập ngày 06/11/2026).

[5] Microsoft Learn, "Primary and foreign key constraints & Check constraints," https://learn.microsoft.com/en-us/sql/relational-databases/tables/primary-and-foreign-key-constraints (truy cập ngày 06/11/2026).

[6] Microsoft Learn, "Transactions and Concurrency in ADO.NET," https://learn.microsoft.com/en-us/dotnet/framework/data/adonet/transactions-and-concurrency (truy cập ngày 06/11/2026).

[7] Microsoft Learn, "Rfc2898DeriveBytes.Pbkdf2 Method & CryptographicOperations.FixedTimeEquals," https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.rfc2898derivebytes.pbkdf2 (truy cập ngày 06/11/2026).

[8] Dapper Lib, "Dapper – a simple object mapper for .NET," https://github.com/DapperLib/Dapper (truy cập ngày 06/11/2026).

[9] Nhóm thực hiện đồ án, "Mã nguồn ứng dụng Quản lý mượn trả thiết bị (QuanLyMuonTra_GD5)," gồm các project `QuanLyMuonTra.App`, `QuanLyMuonTra.UI`, `README.md` và `CHANGELOG.md` (tháng 11/2026).

[10] Nhóm thực hiện đồ án, "Tập lệnh cơ sở dữ liệu và tài liệu kiểm thử bàn giao," các tệp `Database/01_CreateDatabase.sql` đến `07_TestData.sql`, `docs/TEST_CASES.md`, `docs/BAO_CAO_NHOM_TEMPLATE.md` và `docs/SLIDE_OUTLINE.md` trong gói mã nguồn [9].

PHỤ LỤC

Phụ lục A. Liên kết và hồ sơ đồ án

- Repository GitHub của nhóm: https://github.com/viethoang234-se/2611COMP101904-LTW-KhaBi.
- Bản mã nguồn đối chiếu: tệp nén `QuanLyMuonTra_v5_2.zip`, chứa solution `QuanLyMuonTra.slnx`, thư mục `QuanLyMuonTra.App`, `QuanLyMuonTra.UI`, `Database` và `docs`.
- Link video trình diễn (Demo): [NHÓM BỔ SUNG ĐƯỜNG DẪN VIDEO DEMO THỰC TẾ].

Phụ lục B. Kịch bản trình diễn đề xuất

- Bước 1: Đăng nhập bằng tài khoản Nhân viên kho (`kho01` / `Kho@123`), quan sát màn hình Tổng quan (Dashboard F02) và đối chiếu các mục menu được phép hiển thị ở thanh bên trái.
- Bước 2: Mở màn hình Loại thiết bị (F06) để minh họa quyền chỉ xem của nhân viên kho; sau đó mở Quản lý thiết bị (F07) và Người mượn (F08) để thử tìm kiếm, lọc và xem lịch sử mượn.
- Bước 3: Mở màn hình Lập phiếu mượn (F09), chọn người mượn, thêm nhiều thiết bị vào giỏ, thử nhập số lượng vượt tồn khả dụng để xem cảnh báo, sau đó lưu phiếu hợp lệ và kiểm tra tồn kho giảm tương ứng.
- Bước 4: Mở Tra cứu phiếu mượn (F10), nhấp đúp mở Chi tiết phiếu mượn (F11) và thử chức năng Xem trước / In phiếu mượn.
- Bước 5: Mở Ghi nhận trả (F12), thực hiện trả một phần thiết bị ở tình trạng Tốt và một phần ở tình trạng Hư hỏng (có kiểm tra bắt buộc ghi chú); kiểm tra trạng thái phiếu chuyển sang Trả một phần / Đã trả.
- Bước 6: Mở Biên bản trả (F13) và Phiếu quá hạn (F14) để kiểm tra dữ liệu biên bản vừa tạo và danh sách quá hạn từ thủ tục `sp_GetOverdueSlips`.
- Bước 7: Bật/tắt chế độ Giao diện tối (Dark Mode) trên thanh tiêu đề; sau đó nhấn Đăng xuất và đăng nhập bằng tài khoản Người quản lý (`admin` / `Admin@123`).

- Bước 8: Với quyền quản lý, trình diễn Hủy biên bản trả (F13), xem Báo cáo tổng hợp (F15), Thống kê thiết bị mượn nhiều (F18), Quản lý người dùng (F04), Phân quyền nhóm quyền (F05) và Cấu hình mã tự động (F16).

Phụ lục D. Mã nguồn tiêu biểu – Script tạo bảng dữ liệu

Tệp `Database/02_CreateTables.sql`: định nghĩa 12 bảng của cơ sở dữ liệu `QuanLyMuonTra`.

```
/* ============================================================================
   02 - TẠO BẢNG và NÂNG CẤP CỘT
   - Tạo bảng nếu chưa có (CSDL mới).
   - Bổ sung cột mới nếu thiếu (CSDL cũ từ phiên bản trước vẫn nâng cấp được, giữ nguyên dữ liệu).

   Quy ước trạng thái:
   - LoanSlips.Status        : 0 = còn thiết bị chưa trả, 1 = đã trả đủ.
                               "Trả một phần" và "Quá hạn" được TÍNH từ chi tiết + ngày hiện tại (xem vw_LoanSlipSummary).
   - ReturnRecordDetails.ConditionCode : 0 = Tốt, 1 = Hư hỏng, 2 = Mất, 3 = Thiếu phụ kiện.
   - Devices : TotalQuantity = AvailableQuantity + (đang mượn) + DamagedQuantity + LostQuantity.
   ============================================================================ */
USE QuanLyMuonTra;
GO

IF OBJECT_ID(N'dbo.Roles') IS NULL
CREATE TABLE dbo.Roles (
    RoleId      INT IDENTITY(1,1) PRIMARY KEY,
    RoleName    NVARCHAR(50)  NOT NULL UNIQUE,
    Description NVARCHAR(200) NULL
);

IF OBJECT_ID(N'dbo.Permissions') IS NULL
CREATE TABLE dbo.Permissions (
    PermissionId INT IDENTITY(1,1) PRIMARY KEY,
    Code         VARCHAR(50)   NOT NULL UNIQUE,
    Name         NVARCHAR(100) NOT NULL
);

IF OBJECT_ID(N'dbo.RolePermissions') IS NULL
CREATE TABLE dbo.RolePermissions (
    RoleId       INT NOT NULL REFERENCES dbo.Roles(RoleId),
    PermissionId INT NOT NULL REFERENCES dbo.Permissions(PermissionId),
    PRIMARY KEY (RoleId, PermissionId)
);

IF OBJECT_ID(N'dbo.Users') IS NULL
CREATE TABLE dbo.Users (
    UserId       INT IDENTITY(1,1) PRIMARY KEY,
    Username     VARCHAR(50)   NOT NULL UNIQUE,
```

```
    PasswordHash VARCHAR(200)  NOT NULL,
    FullName     NVARCHAR(100) NOT NULL,
    Email        NVARCHAR(100) NULL,
    Phone        NVARCHAR(20)  NULL,
    RoleId       INT NOT NULL REFERENCES dbo.Roles(RoleId),
    IsActive     BIT NOT NULL DEFAULT 1,
    CreatedAt    DATETIME2 NOT NULL DEFAULT SYSDATETIME()
);

IF OBJECT_ID(N'dbo.DeviceCategories') IS NULL
CREATE TABLE dbo.DeviceCategories (
    CategoryId   INT IDENTITY(1,1) PRIMARY KEY,
    CategoryCode VARCHAR(20)   NOT NULL UNIQUE,
    CategoryName NVARCHAR(100) NOT NULL,
    Description  NVARCHAR(255) NULL,
    IsActive     BIT NOT NULL DEFAULT 1
);

IF OBJECT_ID(N'dbo.Devices') IS NULL
CREATE TABLE dbo.Devices (
    DeviceId          INT IDENTITY(1,1) PRIMARY KEY,
    DeviceCode        VARCHAR(30)   NOT NULL UNIQUE,
    DeviceName        NVARCHAR(150) NOT NULL,
    CategoryId        INT NOT NULL REFERENCES
    ↪ dbo.DeviceCategories(CategoryId),
    TotalQuantity     INT NOT NULL DEFAULT 0 CHECK (TotalQuantity >= 0),
    AvailableQuantity INT NOT NULL DEFAULT 0 CHECK (AvailableQuantity >=
    ↪ 0),
    Unit              NVARCHAR(30)  NULL,
    Note              NVARCHAR(255) NULL,
    IsActive          BIT NOT NULL DEFAULT 1,
    CreatedAt         DATETIME2 NOT NULL DEFAULT SYSDATETIME()
);

IF OBJECT_ID(N'dbo.Borrowers') IS NULL
CREATE TABLE dbo.Borrowers (
    BorrowerId   INT IDENTITY(1,1) PRIMARY KEY,
    BorrowerCode VARCHAR(30)   NOT NULL UNIQUE,
    FullName     NVARCHAR(100) NOT NULL,
    Phone        NVARCHAR(20)  NULL,
    Email        NVARCHAR(100) NULL,
    Department   NVARCHAR(100) NULL,
    Note         NVARCHAR(255) NULL,
    IsActive     BIT NOT NULL DEFAULT 1
);

IF OBJECT_ID(N'dbo.LoanSlips') IS NULL
CREATE TABLE dbo.LoanSlips (
    LoanSlipId        INT IDENTITY(1,1) PRIMARY KEY,
    SlipCode          VARCHAR(30) NOT NULL UNIQUE,
    BorrowerId        INT NOT NULL REFERENCES dbo.Borrowers(BorrowerId),
    CreatedBy         INT NOT NULL REFERENCES dbo.Users(UserId),
    BorrowDate        DATE NOT NULL,
    DueDate           DATE NOT NULL,
    Status            TINYINT NOT NULL DEFAULT 0,
    Note              NVARCHAR(255) NULL,
    BorrowerSignature VARBINARY(MAX) NULL,
    CreatedAt         DATETIME2 NOT NULL DEFAULT SYSDATETIME()
```

```
);

IF OBJECT_ID(N'dbo.LoanSlipDetails') IS NULL
CREATE TABLE dbo.LoanSlipDetails (
    DetailId         INT IDENTITY(1,1) PRIMARY KEY,
    LoanSlipId       INT NOT NULL REFERENCES dbo.LoanSlips(LoanSlipId),
    DeviceId         INT NOT NULL REFERENCES dbo.Devices(DeviceId),
    Quantity         INT NOT NULL CHECK (Quantity > 0),
    ReturnedQuantity INT NOT NULL DEFAULT 0
);

IF OBJECT_ID(N'dbo.ReturnRecords') IS NULL
CREATE TABLE dbo.ReturnRecords (
    ReturnRecordId    INT IDENTITY(1,1) PRIMARY KEY,
    RecordCode        VARCHAR(30) NOT NULL UNIQUE,
    LoanSlipId        INT NOT NULL REFERENCES dbo.LoanSlips(LoanSlipId),
    ReturnDate        DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    ReceivedBy        INT NOT NULL REFERENCES dbo.Users(UserId),
    Note              NVARCHAR(255) NULL,
    IsCancelled       BIT NOT NULL DEFAULT 0,
    ReceiverSignature VARBINARY(MAX) NULL,
    BorrowerSignature VARBINARY(MAX) NULL
);

/* Condition (NVARCHAR) giữ nguyên tên cũ, dùng làm GHI CHÚ tình trạng
 ↪  (mô tả hư hỏng, thiếu phụ kiện...). */
IF OBJECT_ID(N'dbo.ReturnRecordDetails') IS NULL
CREATE TABLE dbo.ReturnRecordDetails (
    Id             INT IDENTITY(1,1) PRIMARY KEY,
    ReturnRecordId INT NOT NULL REFERENCES
     ↪  dbo.ReturnRecords(ReturnRecordId),
    DeviceId       INT NOT NULL REFERENCES dbo.Devices(DeviceId),
    Quantity       INT NOT NULL CHECK (Quantity > 0),
    Condition      NVARCHAR(100) NULL
);

/* Mã tự động: ví dụ PM-20261106-0001 (UseDate = 1) hoặc TB0009 (UseDate
 ↪  = 0) */
IF OBJECT_ID(N'dbo.CodeConfigs') IS NULL
CREATE TABLE dbo.CodeConfigs (
    ConfigId      INT IDENTITY(1,1) PRIMARY KEY,
    EntityName    VARCHAR(30)  NOT NULL UNIQUE,
    Prefix        VARCHAR(10)  NOT NULL,
    UseDate       BIT NOT NULL DEFAULT 1,
    NumberLength  INT NOT NULL DEFAULT 4,
    CurrentNumber INT NOT NULL DEFAULT 0,
    ResetDaily    BIT NOT NULL DEFAULT 1,
    LastResetDate DATE NULL
);
GO

/* ---------- NÂNG CẤP: bổ sung cột mới (an toàn khi chạy lại) ----------
 ↪  */
IF COL_LENGTH('dbo.Devices', 'DamagedQuantity') IS NULL
    ALTER TABLE dbo.Devices ADD DamagedQuantity INT NOT NULL CONSTRAINT
     ↪  DF_Devices_Damaged DEFAULT 0;
IF COL_LENGTH('dbo.Devices', 'LostQuantity') IS NULL
```

```
    ALTER TABLE dbo.Devices ADD LostQuantity INT NOT NULL CONSTRAINT
     ↪  DF_Devices_Lost DEFAULT 0;

IF COL_LENGTH('dbo.Borrowers', 'Address') IS NULL
    ALTER TABLE dbo.Borrowers ADD Address NVARCHAR(200) NULL;

IF COL_LENGTH('dbo.ReturnRecords', 'GiverName') IS NULL
    ALTER TABLE dbo.ReturnRecords ADD GiverName NVARCHAR(100) NULL;
IF COL_LENGTH('dbo.ReturnRecords', 'CancelledBy') IS NULL
    ALTER TABLE dbo.ReturnRecords ADD CancelledBy INT NULL;
IF COL_LENGTH('dbo.ReturnRecords', 'CancelledAt') IS NULL
    ALTER TABLE dbo.ReturnRecords ADD CancelledAt DATETIME2 NULL;
IF COL_LENGTH('dbo.ReturnRecords', 'CancelReason') IS NULL
    ALTER TABLE dbo.ReturnRecords ADD CancelReason NVARCHAR(255) NULL;

IF COL_LENGTH('dbo.ReturnRecordDetails', 'LoanSlipDetailId') IS NULL
    ALTER TABLE dbo.ReturnRecordDetails ADD LoanSlipDetailId INT NULL;
IF COL_LENGTH('dbo.ReturnRecordDetails', 'ConditionCode') IS NULL
    ALTER TABLE dbo.ReturnRecordDetails ADD ConditionCode TINYINT NOT
     ↪  NULL CONSTRAINT DF_RRD_ConditionCode DEFAULT 0;
GO

/* Truy vết dữ liệu cũ: gắn dòng trả với dòng mượn khi xác định được duy
 ↪  nhất */
UPDATE rd
SET    rd.LoanSlipDetailId = d.DetailId
FROM   dbo.ReturnRecordDetails rd
JOIN   dbo.ReturnRecords r      ON r.ReturnRecordId = rd.ReturnRecordId
JOIN   dbo.LoanSlipDetails d    ON d.LoanSlipId = r.LoanSlipId AND
 ↪  d.DeviceId = rd.DeviceId
WHERE  rd.LoanSlipDetailId IS NULL
  AND (SELECT COUNT(*) FROM dbo.LoanSlipDetails x
       WHERE x.LoanSlipId = r.LoanSlipId AND x.DeviceId = rd.DeviceId) =
        ↪  1;
GO
```

Phụ lục E. Mã nguồn tiêu biểu – Ràng buộc và chỉ mục

Tệp `Database/03_Constraints.sql`: các ràng buộc `CHECK`, `UNIQUE`, `FOREIGN KEY`.

```
/* ================================================================
   03 - RÀNG BUỘC (PK/FK/UNIQUE/CHECK). Chạy lại an toàn: chỉ thêm ràng
    ↪  buộc còn thiếu.
   ================================================================
    ↪  */
USE QuanLyMuonTra;
GO

/* Tồn kho: 0 <= Available, Damaged, Lost và Available + Damaged + Lost
 ↪  <= Total */
IF NOT EXISTS (SELECT 1 FROM sys.check_constraints WHERE name =
 ↪  'CK_Devices_Inventory' AND parent_object_id =
 ↪  OBJECT_ID('dbo.Devices'))
```

```
    ALTER TABLE dbo.Devices WITH CHECK ADD CONSTRAINT CK_Devices_Inventory CHECK (
        AvailableQuantity >= 0 AND DamagedQuantity >= 0 AND LostQuantity >= 0
        AND AvailableQuantity <= TotalQuantity
        AND AvailableQuantity + DamagedQuantity + LostQuantity <= TotalQuantity);

/* Số lượng đã trả: 0 <= ReturnedQuantity <= Quantity */
IF NOT EXISTS (SELECT 1 FROM sys.check_constraints WHERE name = 'CK_LoanSlipDetails_Returned' AND parent_object_id = OBJECT_ID('dbo.LoanSlipDetails'))
    ALTER TABLE dbo.LoanSlipDetails WITH CHECK ADD CONSTRAINT CK_LoanSlipDetails_Returned CHECK (
        ReturnedQuantity >= 0 AND ReturnedQuantity <= Quantity);

IF NOT EXISTS (SELECT 1 FROM sys.check_constraints WHERE name = 'CK_LoanSlips_Status' AND parent_object_id = OBJECT_ID('dbo.LoanSlips'))
    ALTER TABLE dbo.LoanSlips WITH CHECK ADD CONSTRAINT CK_LoanSlips_Status CHECK (Status IN (0, 1));

/* Hạn trả >= ngày mượn (NOCHECK để không chặn dữ liệu cũ, vẫn áp dụng cho dữ liệu mới) */
IF NOT EXISTS (SELECT 1 FROM sys.check_constraints WHERE name = 'CK_LoanSlips_Dates' AND parent_object_id = OBJECT_ID('dbo.LoanSlips'))
    ALTER TABLE dbo.LoanSlips WITH NOCHECK ADD CONSTRAINT CK_LoanSlips_Dates CHECK (DueDate >= BorrowDate);

IF NOT EXISTS (SELECT 1 FROM sys.check_constraints WHERE name = 'CK_RRD_ConditionCode' AND parent_object_id = OBJECT_ID('dbo.ReturnRecordDetails'))
    ALTER TABLE dbo.ReturnRecordDetails WITH CHECK ADD CONSTRAINT CK_RRD_ConditionCode CHECK (ConditionCode IN (0, 1, 2, 3));

/* Mã tự động: đặt lại theo ngày chỉ hợp lệ khi mã có chứa ngày (tránh trùng mã) */
IF NOT EXISTS (SELECT 1 FROM sys.check_constraints WHERE name = 'CK_CodeConfigs_Rules' AND parent_object_id = OBJECT_ID('dbo.CodeConfigs'))
    ALTER TABLE dbo.CodeConfigs WITH CHECK ADD CONSTRAINT CK_CodeConfigs_Rules CHECK (
        NumberLength BETWEEN 1 AND 9 AND CurrentNumber >= 0 AND (ResetDaily = 0 OR UseDate = 1));

/* Truy vết: dòng trả -> dòng mượn */
IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_RRD_LoanSlipDetail')
    ALTER TABLE dbo.ReturnRecordDetails ADD CONSTRAINT FK_RRD_LoanSlipDetail
        FOREIGN KEY (LoanSlipDetailId) REFERENCES dbo.LoanSlipDetails(DetailId);

/* Người hủy biên bản */
IF NOT EXISTS (SELECT 1 FROM sys.foreign_keys WHERE name = 'FK_ReturnRecords_CancelledBy')
```

```
    ALTER TABLE dbo.ReturnRecords ADD CONSTRAINT FK_ReturnRecords_CancelledBy
        FOREIGN KEY (CancelledBy) REFERENCES dbo.Users(UserId);

/* Một thiết bị chỉ xuất hiện một lần trong cùng phiếu mượn (bỏ qua nếu dữ liệu cũ đang trùng) */
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'UQ_LoanSlipDetails_Slip_Device' AND object_id = OBJECT_ID('dbo.LoanSlipDetails'))
   AND NOT EXISTS (SELECT 1 FROM dbo.LoanSlipDetails GROUP BY LoanSlipId, DeviceId HAVING COUNT(*) > 1)
    CREATE UNIQUE INDEX UQ_LoanSlipDetails_Slip_Device ON dbo.LoanSlipDetails (LoanSlipId, DeviceId);
GO
```

```
/* ============================================================================
   04 - INDEX cho các cột hay tìm kiếm / lọc / nối bảng. Chạy lại an toàn.
   ============================================================================ */
USE QuanLyMuonTra;
GO

IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_Devices_Name' AND object_id = OBJECT_ID('dbo.Devices'))
    CREATE NONCLUSTERED INDEX IX_Devices_Name ON dbo.Devices (DeviceName);
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_Devices_Category' AND object_id = OBJECT_ID('dbo.Devices'))
    CREATE NONCLUSTERED INDEX IX_Devices_Category ON dbo.Devices (CategoryId, IsActive) INCLUDE (AvailableQuantity, TotalQuantity);
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_Borrowers_Name' AND object_id = OBJECT_ID('dbo.Borrowers'))
    CREATE NONCLUSTERED INDEX IX_Borrowers_Name ON dbo.Borrowers (FullName);
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_Borrowers_Phone' AND object_id = OBJECT_ID('dbo.Borrowers'))
    CREATE NONCLUSTERED INDEX IX_Borrowers_Phone ON dbo.Borrowers (Phone);
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_LoanSlips_Borrower' AND object_id = OBJECT_ID('dbo.LoanSlips'))
    CREATE NONCLUSTERED INDEX IX_LoanSlips_Borrower ON dbo.LoanSlips (BorrowerId);
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_LoanSlips_BorrowDate' AND object_id = OBJECT_ID('dbo.LoanSlips'))
    CREATE NONCLUSTERED INDEX IX_LoanSlips_BorrowDate ON dbo.LoanSlips (BorrowDate);
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_LoanSlips_Due_Status' AND object_id = OBJECT_ID('dbo.LoanSlips'))
    CREATE NONCLUSTERED INDEX IX_LoanSlips_Due_Status ON dbo.LoanSlips (DueDate, Status);
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_LoanSlipDetails_Slip' AND object_id = OBJECT_ID('dbo.LoanSlipDetails'))
    CREATE NONCLUSTERED INDEX IX_LoanSlipDetails_Slip ON dbo.LoanSlipDetails (LoanSlipId) INCLUDE (DeviceId, Quantity, ReturnedQuantity);
```

```
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_LoanSlipDetails_Device' AND object_id = OBJECT_ID('dbo.LoanSlipDetails'))
    CREATE NONCLUSTERED INDEX IX_LoanSlipDetails_Device ON dbo.LoanSlipDetails (DeviceId) INCLUDE (Quantity, ReturnedQuantity);
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_ReturnRecords_Slip' AND object_id = OBJECT_ID('dbo.ReturnRecords'))
    CREATE NONCLUSTERED INDEX IX_ReturnRecords_Slip ON dbo.ReturnRecords (LoanSlipId);
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_ReturnRecords_Date' AND object_id = OBJECT_ID('dbo.ReturnRecords'))
    CREATE NONCLUSTERED INDEX IX_ReturnRecords_Date ON dbo.ReturnRecords (ReturnDate);
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_RRD_Record' AND object_id = OBJECT_ID('dbo.ReturnRecordDetails'))
    CREATE NONCLUSTERED INDEX IX_RRD_Record ON dbo.ReturnRecordDetails (ReturnRecordId);
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_RRD_Device' AND object_id = OBJECT_ID('dbo.ReturnRecordDetails'))
    CREATE NONCLUSTERED INDEX IX_RRD_Device ON dbo.ReturnRecordDetails (DeviceId);
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name = 'IX_RRD_LoanSlipDetail' AND object_id = OBJECT_ID('dbo.ReturnRecordDetails'))
    CREATE NONCLUSTERED INDEX IX_RRD_LoanSlipDetail ON dbo.ReturnRecordDetails (LoanSlipDetailId);
GO
```

Phụ lục F. Mã nguồn tiêu biểu – View và Stored procedure

Tệp `Database/05_StoredProcedures.sql`: view tổng hợp và các thủ tục lưu trữ (kể cả `sp_GetOverdueSlips`).

```
/* ============================================================================
   05 - VIEW và STORED PROCEDURE (CREATE OR ALTER: chạy lại an toàn)
   ============================================================================ */
USE QuanLyMuonTra;
GO

/* ----------------------------------------------------------------------------
   vw_LoanSlipSummary - tổng hợp phiếu mượn + trạng thái hiển thị (tính tại thời điểm truy vấn)
   DisplayStatus: 0 = Đang mượn, 1 = Trả một phần, 2 = Đã trả, 3 = Quá hạn
   (Quá hạn = còn thiết bị chưa trả và đã qua hạn trả; ưu tiên hơn "Trả một phần")
   ---------------------------------------------------------------------------- */
CREATE OR ALTER VIEW dbo.vw_LoanSlipSummary
AS
```

```
SELECT  s.LoanSlipId, s.SlipCode, s.BorrowerId, b.BorrowerCode, b.FullName AS BorrowerName,
        ISNULL(b.Phone, N'') AS Phone, s.CreatedBy, u.FullName AS CreatedByName,
        s.BorrowDate, s.DueDate, s.Status, ISNULL(s.Note, N'') AS Note, s.CreatedAt,
        ISNULL(t.TotalQty, 0)    AS TotalQty,
        ISNULL(t.ReturnedQty, 0) AS ReturnedQty,
        ISNULL(t.TotalQty, 0) - ISNULL(t.ReturnedQty, 0) AS RemainingQty,
        CASE WHEN ISNULL(t.TotalQty, 0) - ISNULL(t.ReturnedQty, 0) = 0 THEN 2
             WHEN s.DueDate < CAST(SYSDATETIME() AS DATE) THEN 3
             WHEN ISNULL(t.ReturnedQty, 0) > 0 THEN 1
             ELSE 0 END AS DisplayStatus,
        CASE WHEN ISNULL(t.TotalQty, 0) - ISNULL(t.ReturnedQty, 0) > 0
              AND s.DueDate < CAST(SYSDATETIME() AS DATE)
             THEN DATEDIFF(DAY, s.DueDate, CAST(SYSDATETIME() AS DATE))
             ELSE 0 END AS DaysOverdue
FROM    dbo.LoanSlips s
JOIN    dbo.Borrowers b ON b.BorrowerId = s.BorrowerId
JOIN    dbo.Users u     ON u.UserId = s.CreatedBy
OUTER APPLY (SELECT SUM(d.Quantity) AS TotalQty, SUM(d.ReturnedQuantity) AS ReturnedQty
             FROM dbo.LoanSlipDetails d WHERE d.LoanSlipId = s.LoanSlipId) t;
GO

/* ---------------------------------------------------------------------
   sp_GetNextCode - sinh mã tự động theo CodeConfigs.
   Khóa dòng cấu hình (UPDLOCK, HOLDLOCK) nên nhiều người cùng thao tác không bao giờ trùng mã.
   Gọi trong cùng transaction với việc lưu phiếu: nếu rollback thì số cũng được hoàn lại.
   @Peek = 1: chỉ xem mã kế tiếp, không tăng bộ đếm (dùng để gợi ý trên giao diện).
   --------------------------------------------------------------------- */
CREATE OR ALTER PROCEDURE dbo.sp_GetNextCode
    @EntityName VARCHAR(30),
    @Code       VARCHAR(50) OUTPUT,
    @Peek       BIT = 0
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @today DATE = CAST(SYSDATETIME() AS DATE);
    DECLARE @prefix VARCHAR(10), @useDate BIT, @len INT, @num INT, @reset BIT, @last DATE;

    SELECT @prefix = Prefix, @useDate = UseDate, @len = NumberLength,
           @num = CurrentNumber, @reset = ResetDaily, @last = LastResetDate
    FROM   dbo.CodeConfigs WITH (UPDLOCK, HOLDLOCK)
    WHERE  EntityName = @EntityName;

    IF @prefix IS NULL
        THROW 50001, N'Chưa cấu hình mã tự động cho đối tượng này.', 1;

    IF @reset = 1 AND (@last IS NULL OR @last <> @today) SET @num = 0;
```

```
    SET @num = @num + 1;

    IF @Peek = 0
        UPDATE dbo.CodeConfigs SET CurrentNumber = @num, LastResetDate =
        ↪ @today WHERE EntityName = @EntityName;

    DECLARE @numText VARCHAR(20) = CAST(@num AS VARCHAR(20));
    IF LEN(@numText) < @len SET @numText = RIGHT(REPLICATE('0', @len) +
    ↪ @numText, @len);

    SET @Code = @prefix
              + CASE WHEN @useDate = 1 THEN '-' + CONVERT(VARCHAR(8),
              ↪ @today, 112) + '-' ELSE '' END
              + @numText;
END
GO

/* ---------------------------------------------------------------------
   sp_GetOverdueSlips (F14) - danh sách phiếu quá hạn, tính theo ngày
   ↪ hiện tại và số lượng chưa trả thực tế.
   @Keyword: tìm theo mã phiếu, tên người mượn, số điện thoại.
   @MinDaysOverdue: chỉ lấy phiếu quá hạn từ N ngày trở lên.
   @AsOfDate: mặc định là hôm nay.
   ---------------------------------------------------------------------
   ↪ */
CREATE OR ALTER PROCEDURE dbo.sp_GetOverdueSlips
    @Keyword        NVARCHAR(100) = NULL,
    @MinDaysOverdue INT           = NULL,
    @AsOfDate       DATE          = NULL
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @asOf DATE = ISNULL(@AsOfDate, CAST(SYSDATETIME() AS DATE));
    DECLARE @kw NVARCHAR(100) = NULLIF(LTRIM(RTRIM(@Keyword)), N'');
    DECLARE @like NVARCHAR(110) = CASE WHEN @kw IS NULL THEN NULL ELSE
    ↪ N'%' + @kw + N'%' END;

    SELECT  s.LoanSlipId, s.SlipCode, b.BorrowerId, b.FullName AS
    ↪ BorrowerName, ISNULL(b.Phone, N'') AS Phone,
            s.BorrowDate, s.DueDate,
            DATEDIFF(DAY, s.DueDate, @asOf) AS DaysOverdue,
            x.UnreturnedQty, x.UnreturnedDevices
    FROM    dbo.LoanSlips s
    JOIN    dbo.Borrowers b ON b.BorrowerId = s.BorrowerId
    CROSS APPLY (
        SELECT  SUM(d.Quantity - d.ReturnedQuantity) AS UnreturnedQty,
                STRING_AGG(CONVERT(NVARCHAR(MAX), dv.DeviceName + N' (x'
                ↪ + CAST(d.Quantity - d.ReturnedQuantity AS
                ↪ NVARCHAR(10)) + N')'), N', ')
                    WITHIN GROUP (ORDER BY dv.DeviceName) AS
                    ↪ UnreturnedDevices
        FROM    dbo.LoanSlipDetails d
        JOIN    dbo.Devices dv ON dv.DeviceId = d.DeviceId
        WHERE   d.LoanSlipId = s.LoanSlipId AND d.ReturnedQuantity <
        ↪ d.Quantity
    ) x
    WHERE   s.DueDate < @asOf
        AND ISNULL(x.UnreturnedQty, 0) > 0
```

```
        AND (@MinDaysOverdue IS NULL OR DATEDIFF(DAY, s.DueDate, @asOf)
        ↪  >= @MinDaysOverdue)
        AND (@like IS NULL OR s.SlipCode LIKE @like OR b.FullName LIKE
        ↪  @like OR b.Phone LIKE @like)
    ORDER BY DATEDIFF(DAY, s.DueDate, @asOf) DESC, s.SlipCode;
END
GO

/* ----------------------------------------------------------------------
   sp_RecalculateInventory - đồng bộ lại trạng thái phiếu và tồn kho khả
   ↪  dụng:
   Available = Total - (đang mượn) - Damaged - Lost  (không âm).
   Dùng sau khi nạp dữ liệu mẫu hoặc khi cần sửa sai lệch. An toàn khi
   ↪  chạy nhiều lần.
   ----------------------------------------------------------------------
   ↪  */
CREATE OR ALTER PROCEDURE dbo.sp_RecalculateInventory
AS
BEGIN
    SET NOCOUNT ON;

    UPDATE s
    SET    Status = CASE WHEN EXISTS (SELECT 1 FROM dbo.LoanSlipDetails d
    ↪  WHERE d.LoanSlipId = s.LoanSlipId)
                         AND NOT EXISTS (SELECT 1 FROM
                         ↪  dbo.LoanSlipDetails d
                                        WHERE d.LoanSlipId =
                                        ↪  s.LoanSlipId AND
                                        ↪  d.ReturnedQuantity <
                                        ↪  d.Quantity)
                        THEN 1 ELSE 0 END
    FROM   dbo.LoanSlips s;

    ;WITH onloan AS (
        SELECT DeviceId, SUM(Quantity - ReturnedQuantity) AS Qty
        FROM   dbo.LoanSlipDetails GROUP BY DeviceId
    )
    UPDATE dv
    SET    AvailableQuantity =
           CASE WHEN dv.TotalQuantity - ISNULL(o.Qty, 0) -
           ↪  dv.DamagedQuantity - dv.LostQuantity < 0 THEN 0
                ELSE dv.TotalQuantity - ISNULL(o.Qty, 0) -
                ↪  dv.DamagedQuantity - dv.LostQuantity END
    FROM   dbo.Devices dv
    LEFT JOIN onloan o ON o.DeviceId = dv.DeviceId;
END
GO
```

Phụ lục G. Mã nguồn tiêu biểu – Kết nối CSDL và transaction

Tệp `QuanLyMuonTra.App/Data/Db.cs`: tạo kết nối, chạy transaction và tự thử lại khi gặp deadlock.

```
using System.Data;
using Microsoft.Data.SqlClient;
```

```
3
4  namespace QuanLyMuonTra.App.Data;
5
6  /// <summary>Lỗi nghiệp vụ có thông báo hiển thị cho người dùng. Ném bên
   ↪  trong transaction để rollback toàn bộ.</summary>
7  public sealed class BusinessException : Exception
8  {
9      public BusinessException(string message) : base(message) { }
10 }
11
12 /// <summary>Kết nối SQL Server. Chuỗi kết nối đọc từ file connection.txt
   ↪  cạnh file .exe.</summary>
13 public static class Db
14 {
15     const string DefaultConnectionString =
16         "Server=localhost;Database=QuanLyMuonTra;Trusted_Connection=True
           ↪  ;TrustServerCertificate=True;";
17
18     public static string ConnectionString { get; } = Load();
19
20     static string Load()
21     {
22         var file = Path.Combine(AppContext.BaseDirectory,
           ↪  "connection.txt");
23         if (File.Exists(file))
24         {
25             var text = File.ReadAllText(file).Trim();
26             if (text.Length > 0) return text;
27         }
28         return DefaultConnectionString;
29     }
30
31     public static SqlConnection Open()
32     {
33         var connection = new SqlConnection(ConnectionString);
34         connection.Open();
35         return connection;
36     }
37
38     /// <summary>
39     /// Chạy một nghiệp vụ trong SQL transaction: lỗi bất kỳ -> rollback
       ↪  toàn bộ.
40     /// Tự thử lại tối đa 3 lần khi SQL Server chọn giao dịch này làm nạn
       ↪  nhân deadlock (mã lỗi 1205).
41     /// </summary>
42     public static T InTransaction<T>(Func<SqlConnection, SqlTransaction,
       ↪  T> work)
43     {
44         const int maxAttempts = 3;
45         for (int attempt = 1; ; attempt++)
46         {
47             using var conn = Open();
48             using var tx =
               ↪  conn.BeginTransaction(IsolationLevel.ReadCommitted);
49             try
50             {
51                 var result = work(conn, tx);
52                 tx.Commit();
```

```
53                  return result;
54              }
55              catch (SqlException ex) when (ex.Number == 1205 && attempt <
                ↪  maxAttempts)
56              {
57                  TryRollback(tx);
58                  Thread.Sleep(100 * attempt);
59              }
60              catch
61              {
62                  TryRollback(tx);
63                  throw;
64              }
65          }
66      }
67
68      static void TryRollback(SqlTransaction tx)
69      {
70          try { tx.Rollback(); } catch { /* giao dịch đã bị SQL Server hủy
            ↪  */ }
71      }
72
73      /// <summary>Chuyển ngoại lệ thành thông báo tiếng Việt dễ hiểu cho
        ↪  người dùng.</summary>
74      public static string Describe(Exception ex)
75      {
76          if (ex is BusinessException) return ex.Message;
77          if (ex is SqlException sql)
78          {
79              return sql.Number switch
80              {
81                  -2 => "Hết thời gian chờ SQL Server. Vui lòng thử lại.",
82                  -1 or 2 or 53 or 233 or 258 or 1231 or 10053 or 10054 or
                    ↪  10060 or 40 =>
83                      "Không kết nối được tới SQL Server. Kiểm tra SQL
                        ↪  Server đang chạy và chuỗi kết nối trong
                        ↪  connection.txt.",
84                  4060 => "Không mở được cơ sở dữ liệu QuanLyMuonTra. Hãy
                    ↪  chạy các script trong thư mục Database.",
85                  18456 => "SQL Server từ chối đăng nhập. Kiểm tra tài
                    ↪  khoản trong connection.txt.",
86                  1205 => "Hệ thống đang bận xử lý đồng thời. Vui lòng thử
                    ↪  lại.",
87                  2601 or 2627 => "Dữ liệu bị trùng (mã đã tồn tại).",
88                  547 => "Dữ liệu vi phạm ràng buộc của cơ sở dữ liệu (số
                    ↪  lượng không hợp lệ hoặc đang được sử dụng).",
89                  _ => "Lỗi cơ sở dữ liệu: " + sql.Message,
90              };
91          }
92          if (ex is InvalidOperationException &&
            ↪  ex.Message.Contains("connection",
            ↪  StringComparison.OrdinalIgnoreCase))
93              return "Không kết nối được tới cơ sở dữ liệu.";
94          return "Đã xảy ra lỗi: " + ex.Message;
95      }
96  }
```

Phụ lục H. Mã nguồn tiêu biểu – Lập phiếu mượn (tầng Data)

Tệp `QuanLyMuonTra.App/Data/LoanRepository.cs`: trừ tồn kho có điều kiện trong transaction.

```
using Dapper;
using Microsoft.Data.SqlClient;
using QuanLyMuonTra.App.Models;

namespace QuanLyMuonTra.App.Data;

public static class LoanRepository
{
    sealed class StockInfo
    {
        public string DeviceName { get; set; } = "";
        public int AvailableQuantity { get; set; }
        public bool IsActive { get; set; }
    }

    /// <summary>
    /// Lưu phiếu mượn + chi tiết + trừ tồn kho trong MỘT transaction.
    /// Việc trừ tồn là câu UPDATE có điều kiện (AvailableQuantity >= số
    ↪  mượn) nên hai người cùng mượn
    /// thiết bị tồn kho thấp thì chỉ một người thành công, người kia
    ↪  nhận thông báo và toàn bộ phiếu bị rollback.
    /// </summary>
    public static (int LoanSlipId, string SlipCode) Create(NewLoanSlip s,
    ↪  int userId) =>
        Db.InTransaction((conn, tx) =>
        {
            int borrowerOk = conn.ExecuteScalar<int>(
                "SELECT COUNT(*) FROM Borrowers WHERE BorrowerId =
                ↪  @BorrowerId AND IsActive = 1", new { s.BorrowerId },
                ↪  tx);
            if (borrowerOk == 0)
                throw new BusinessException("Người mượn không tồn tại
                ↪  hoặc đã ngừng hoạt động.");

            string code = CodeRepository.Next(conn, tx,
            ↪  CodeRepository.LoanSlip);

            int slipId = conn.ExecuteScalar<int>(
                @"INSERT INTO LoanSlips (SlipCode, BorrowerId, CreatedBy,
                ↪  BorrowDate, DueDate, Status, Note)
                  OUTPUT INSERTED.LoanSlipId
                  VALUES (@code, @BorrowerId, @userId, @BorrowDate,
                  ↪  @DueDate, 0, @Note)",
                new { code, s.BorrowerId, userId, BorrowDate =
                ↪  s.BorrowDate.Date, DueDate = s.DueDate.Date, s.Note
                ↪  }, tx);

            // Gộp trùng thiết bị và xử lý theo thứ tự DeviceId cố định
            ↪  để tránh deadlock giữa các phiếu.
            foreach (var group in s.Items.GroupBy(i =>
            ↪  i.DeviceId).OrderBy(g => g.Key))
```

```
39              {
40                  int deviceId = group.Key;
41                  int qty = group.Sum(i => i.Quantity);
42
43                  int rows = conn.Execute(
44                      @"UPDATE Devices SET AvailableQuantity =
                        ↪  AvailableQuantity - @qty
45                        WHERE DeviceId = @deviceId AND IsActive = 1 AND
                          ↪  AvailableQuantity >= @qty",
46                      new { qty, deviceId }, tx);
47
48                  if (rows == 0)
49                  {
50                      var info = conn.QuerySingleOrDefault<StockInfo>(
51                          "SELECT DeviceName, AvailableQuantity, IsActive
                            ↪  FROM Devices WHERE DeviceId = @deviceId",
52                          new { deviceId }, tx);
53                      if (info is null || !info.IsActive)
54                          throw new BusinessException("Có thiết bị không
                            ↪  còn tồn tại hoặc đã ngừng sử dụng.");
55                      throw new BusinessException(
56                          $"Thiết bị \"{info.DeviceName}\" chỉ còn
                            ↪  {info.AvailableQuantity} khả dụng, không đủ
                            ↪  để mượn {qty}.");
57                  }
58
59                  conn.Execute(
60                      "INSERT INTO LoanSlipDetails (LoanSlipId, DeviceId,
                        ↪  Quantity, ReturnedQuantity) VALUES (@slipId,
                        ↪  @deviceId, @qty, 0)",
61                      new { slipId, deviceId, qty }, tx);
62              }
63
64              return (slipId, code);
65          });
66
67      public static List<LoanSlipRow> Search(LoanFilter f)
68      {
69          string? kw = string.IsNullOrWhiteSpace(f.Keyword) ? null :
            ↪  f.Keyword.Trim();
70          string? like = kw is null ? null : $"%{kw}%";
71
72          using var conn = Db.Open();
73          return conn.Query<LoanSlipRow>(
74              @"SELECT * FROM vw_LoanSlipSummary
75                WHERE (@like IS NULL OR SlipCode LIKE @like OR BorrowerName
                  ↪  LIKE @like OR BorrowerCode LIKE @like OR Phone LIKE
                  ↪  @like)
76                  AND (@borrowFrom IS NULL OR BorrowDate >= @borrowFrom)
77                  AND (@borrowTo   IS NULL OR BorrowDate <= @borrowTo)
78                  AND (@dueFrom    IS NULL OR DueDate >= @dueFrom)
79                  AND (@dueTo      IS NULL OR DueDate <= @dueTo)
80                  AND (@status     IS NULL OR DisplayStatus = @status)
81                  AND (@borrowerId IS NULL OR BorrowerId = @borrowerId)
82                ORDER BY LoanSlipId DESC",
83              new
84              {
85                  like,
```

```
86                  borrowFrom = f.BorrowFrom?.Date,
87                  borrowTo = f.BorrowTo?.Date,
88                  dueFrom = f.DueFrom?.Date,
89                  dueTo = f.DueTo?.Date,
90                  status = f.Status,
91                  borrowerId = f.BorrowerId,
92              }).ToList();
93      }
94
95      public static LoanSlipRow? GetHeader(int loanSlipId)
96      {
97          using var conn = Db.Open();
98          return GetHeader(conn, null, loanSlipId);
99      }
100
101     public static LoanSlipRow? GetHeader(SqlConnection conn,
        ↪  SqlTransaction? tx, int loanSlipId) =>
102         conn.QuerySingleOrDefault<LoanSlipRow>(
103             "SELECT * FROM vw_LoanSlipSummary WHERE LoanSlipId =
                ↪  @loanSlipId", new { loanSlipId }, tx);
104
105     public static LoanSlipRow? GetByCode(string slipCode)
106     {
107         using var conn = Db.Open();
108         return conn.QuerySingleOrDefault<LoanSlipRow>(
109             "SELECT * FROM vw_LoanSlipSummary WHERE SlipCode =
                ↪  @slipCode", new { slipCode });
110     }
111
112     public static List<LoanSlipDetailRow> GetDetails(int loanSlipId)
113     {
114         using var conn = Db.Open();
115         return GetDetails(conn, loanSlipId);
116     }
117
118     static List<LoanSlipDetailRow> GetDetails(SqlConnection conn, int
        ↪  loanSlipId) =>
119         conn.Query<LoanSlipDetailRow>(
120             @"SELECT d.DetailId, d.DeviceId, dv.DeviceCode,
                ↪  dv.DeviceName, ISNULL(dv.Unit, N'') AS Unit,
121                      d.Quantity, d.ReturnedQuantity
122               FROM LoanSlipDetails d
123               JOIN Devices dv ON dv.DeviceId = d.DeviceId
124               WHERE d.LoanSlipId = @loanSlipId
125               ORDER BY d.DetailId", new { loanSlipId }).ToList();
126
127     /// <summary>Phiếu đầy đủ: đầu phiếu + thiết bị + lịch sử các lần trả
        ↪  (F11, in phiếu).</summary>
128     public static LoanSlipFull? GetFull(int loanSlipId)
129     {
130         using var conn = Db.Open();
131         var header = GetHeader(conn, null, loanSlipId);
132         if (header is null) return null;
133         return new LoanSlipFull
134         {
135             Header = header,
136             Details = GetDetails(conn, loanSlipId),
137             Returns = ReturnRepository.GetFullByLoan(conn, loanSlipId),
```

```
        };
    }

    /// <summary>Phiếu còn thiết bị chưa trả (dùng cho F12 chọn phiếu để trả).</summary>
    public static List<LoanSlipRow> SearchOpen(string? keyword)
    {
        string? kw = string.IsNullOrWhiteSpace(keyword) ? null : keyword.Trim();
        string? like = kw is null ? null : $"%{kw}%";
        using var conn = Db.Open();
        return conn.Query<LoanSlipRow>(
            @"SELECT * FROM vw_LoanSlipSummary
              WHERE RemainingQty > 0
                AND (@like IS NULL OR SlipCode LIKE @like OR BorrowerName LIKE @like OR BorrowerCode LIKE @like OR Phone LIKE @like)
              ORDER BY DueDate, LoanSlipId", new { like }).ToList();
    }
}
```

Phụ lục I. Mã nguồn tiêu biểu – Ghi nhận trả và hủy biên bản (tầng Data)

Tệp `QuanLyMuonTra.App/Data/ReturnRepository.cs`: ghi nhận trả theo tình trạng và hoàn tác khi hủy biên bản.

```
using Dapper;
using Microsoft.Data.SqlClient;
using QuanLyMuonTra.App.Models;

namespace QuanLyMuonTra.App.Data;

public static class ReturnRepository
{
    sealed class SlipLock
    {
        public int LoanSlipId { get; set; }
        public DateTime BorrowDate { get; set; }
    }

    sealed class DetailLock
    {
        public int DetailId { get; set; }
        public int DeviceId { get; set; }
        public int Quantity { get; set; }
        public int ReturnedQuantity { get; set; }
    }

    sealed class RecordDetailLock
    {
        public int Id { get; set; }
        public int DeviceId { get; set; }
        public int? LoanSlipDetailId { get; set; }
        public int Quantity { get; set; }
```

```
29          public int ConditionCode { get; set; }
30      }
31
32      /// <summary>Nhóm tồn kho mà một số lượng trả sẽ đi vào: (khả dụng,
        ↪  hư hỏng, mất).</summary>
33      static (int Good, int Damaged, int Lost) Bucket(int conditionCode,
        ↪  int qty) => (ReturnCondition)conditionCode switch
34      {
35          ReturnCondition.Good => (qty, 0, 0),
36          ReturnCondition.Lost => (0, 0, qty),
37          _ => (0, qty, 0), // Hư hỏng, Thiếu phụ kiện
38      };
39
40      const string RecomputeStatusSql =
41          @"UPDATE LoanSlips
42            SET Status = CASE WHEN EXISTS (SELECT 1 FROM LoanSlipDetails
              ↪  WHERE LoanSlipId = @loanSlipId AND ReturnedQuantity <
              ↪  Quantity)
43                            THEN 0 ELSE 1 END
44            WHERE LoanSlipId = @loanSlipId";
45
46      static string DeviceName(SqlConnection conn, SqlTransaction tx, int
        ↪  deviceId) =>
47          conn.ExecuteScalar<string?>("SELECT DeviceName FROM Devices WHERE
            ↪  DeviceId = @deviceId", new { deviceId }, tx) ?? "?";
48
49      /// <summary>
50      /// Ghi nhận trả trong MỘT transaction: khóa phiếu -> kiểm tra từng
        ↪  dòng thuộc phiếu và không vượt số còn phải trả ->
51      /// lưu biên bản -> cập nhật ReturnedQuantity -> cập nhật tồn kho
        ↪  theo tình trạng -> cập nhật trạng thái phiếu.
52      /// </summary>
53      public static (int ReturnRecordId, string RecordCode)
        ↪  Create(NewReturnRecord r, int userId) =>
54          Db.InTransaction((conn, tx) =>
55          {
56              var slip = conn.QuerySingleOrDefault<SlipLock>(
57                  "SELECT LoanSlipId, BorrowDate FROM LoanSlips WITH
                    ↪  (UPDLOCK, HOLDLOCK) WHERE LoanSlipId = @LoanSlipId",
58                  new { r.LoanSlipId }, tx);
59              if (slip is null) throw new BusinessException("Phiếu mượn
                ↪  không tồn tại.");
60
61              var details = conn.Query<DetailLock>(
62                  @"SELECT DetailId, DeviceId, Quantity, ReturnedQuantity
63                    FROM LoanSlipDetails WITH (UPDLOCK) WHERE LoanSlipId =
                      ↪  @LoanSlipId",
64                  new { r.LoanSlipId }, tx).ToDictionary(d => d.DetailId);
65
66              if (details.Values.All(d => d.ReturnedQuantity >=
                ↪  d.Quantity))
67                  throw new BusinessException("Phiếu mượn này đã trả đủ,
                    ↪  không còn thiết bị cần trả.");
68              if (r.ReturnDate.Date < slip.BorrowDate.Date)
69                  throw new BusinessException("Ngày trả không được trước
                    ↪  ngày mượn.");
70
71              foreach (var g in r.Items.GroupBy(i => i.LoanSlipDetailId))
```

```
72              {
73                  if (!details.TryGetValue(g.Key, out var d))
74                      throw new BusinessException("Có thiết bị không thuộc
                        ↪  phiếu mượn này.");
75                  int remaining = d.Quantity - d.ReturnedQuantity;
76                  int sum = g.Sum(i => i.Quantity);
77                  if (sum > remaining)
78                      throw new BusinessException(
79                          $"Thiết bị \"{DeviceName(conn, tx,
                            ↪  d.DeviceId)}\": số lượng trả ({sum}) vượt quá
                            ↪  số còn phải trả ({remaining}).");
80              }
81
82              string code = CodeRepository.Next(conn, tx,
                ↪  CodeRepository.ReturnRecord);
83
84              int recordId = conn.ExecuteScalar<int>(
85                  @"INSERT INTO ReturnRecords (RecordCode, LoanSlipId,
                    ↪  ReturnDate, ReceivedBy, GiverName, Note)
86                    OUTPUT INSERTED.ReturnRecordId
87                    VALUES (@code, @LoanSlipId, @ReturnDate, @userId,
                      ↪  @GiverName, @Note)",
88                  new { code, r.LoanSlipId, r.ReturnDate, userId,
                    ↪  r.GiverName, r.Note }, tx);
89
90              // Dòng chi tiết biên bản (mỗi tình trạng một dòng)
91              foreach (var item in r.Items)
92              {
93                  var d = details[item.LoanSlipDetailId];
94                  conn.Execute(
95                      @"INSERT INTO ReturnRecordDetails (ReturnRecordId,
                        ↪  DeviceId, LoanSlipDetailId, Quantity,
                        ↪  ConditionCode, Condition)
96                        VALUES (@recordId, @DeviceId, @DetailId, @Quantity,
                          ↪  @code, @note)",
97                      new { recordId, d.DeviceId, d.DetailId,
                        ↪  item.Quantity, code = (int)item.Condition, note =
                        ↪  item.ConditionNote }, tx);
98              }
99
100             // Cập nhật số lượng đã trả của từng dòng mượn
101             foreach (var g in r.Items.GroupBy(i =>
                ↪  i.LoanSlipDetailId).OrderBy(g => g.Key))
102             {
103                 int qty = g.Sum(i => i.Quantity);
104                 int rows = conn.Execute(
105                     @"UPDATE LoanSlipDetails SET ReturnedQuantity =
                       ↪  ReturnedQuantity + @qty
106                       WHERE DetailId = @id AND ReturnedQuantity + @qty <=
                         ↪  Quantity",
107                     new { qty, id = g.Key }, tx);
108                 if (rows == 0) throw new BusinessException("Số lượng trả
                    ↪  không hợp lệ (dữ liệu vừa thay đổi). Vui lòng tải lại
                    ↪  phiếu.");
109             }
110
111             // Cập nhật tồn kho theo tình trạng, theo thứ tự DeviceId cố
                ↪  định
```

```
            var perDevice = r.Items
                .Select(i => (DeviceId:
                ↪ details[i.LoanSlipDetailId].DeviceId, b:
                ↪ Bucket((int)i.Condition, i.Quantity)))
                .GroupBy(x => x.DeviceId).OrderBy(g => g.Key);
            foreach (var g in perDevice)
            {
                int good = g.Sum(x => x.b.Good), damaged = g.Sum(x =>
                ↪ x.b.Damaged), lost = g.Sum(x => x.b.Lost);
                int rows = conn.Execute(
                    @"UPDATE Devices
                      SET AvailableQuantity = AvailableQuantity + @good,
                          DamagedQuantity   = DamagedQuantity + @damaged,
                          LostQuantity      = LostQuantity + @lost
                      WHERE DeviceId = @deviceId",
                    new { good, damaged, lost, deviceId = g.Key }, tx);
                if (rows == 0) throw new BusinessException("Thiết bị
                ↪ trong phiếu không còn tồn tại.");
            }

            conn.Execute(RecomputeStatusSql, new { r.LoanSlipId }, tx);
            return (recordId, code);
        });

    /// <summary>
    /// Hủy biên bản trả (không xóa dữ liệu): hoàn tác số lượng đã trả và
    ↪ tồn kho đúng nhóm đã cộng lúc trả.
    /// Nếu tồn kho hiện tại không đủ để hoàn tác (thiết bị đã được cho
    ↪ mượn lại hoặc đã sửa xong) thì từ chối.
    /// </summary>
    public static void Cancel(int returnRecordId, int userId, string
    ↪ reason) =>
        Db.InTransaction((conn, tx) =>
        {
            int? slipId = conn.QuerySingleOrDefault<int?>(
                "SELECT LoanSlipId FROM ReturnRecords WHERE
                ↪ ReturnRecordId = @returnRecordId", new {
                ↪ returnRecordId }, tx);
            if (slipId is null) throw new BusinessException("Biên bản
            ↪ không tồn tại.");

            // Khóa phiếu trước (cùng thứ tự khóa với Create) rồi mới đọc
            ↪ lại trạng thái biên bản
            conn.ExecuteScalar<int?>(
                "SELECT LoanSlipId FROM LoanSlips WITH (UPDLOCK,
                ↪ HOLDLOCK) WHERE LoanSlipId = @slipId", new { slipId
                ↪ }, tx);

            bool cancelled = conn.ExecuteScalar<bool>(
                "SELECT IsCancelled FROM ReturnRecords WHERE
                ↪ ReturnRecordId = @returnRecordId", new {
                ↪ returnRecordId }, tx);
            if (cancelled) throw new BusinessException("Biên bản này đã
            ↪ được hủy trước đó.");

            var lines = conn.Query<RecordDetailLock>(
                @"SELECT Id, DeviceId, LoanSlipDetailId, Quantity,
                ↪ ConditionCode
```

```
153                   FROM ReturnRecordDetails WHERE ReturnRecordId =
                        ↪ @returnRecordId",
154                   new { returnRecordId }, tx).ToList();
155
156               foreach (var line in lines)
157               {
158                   int? detailId = line.LoanSlipDetailId ??
                        ↪ conn.QuerySingleOrDefault<int?>(
159                       @"SELECT MIN(DetailId) FROM LoanSlipDetails
160                         WHERE LoanSlipId = @slipId AND DeviceId =
                            ↪ @DeviceId", new { slipId, line.DeviceId }, tx);
161                   if (detailId is null)
162                       throw new BusinessException("Không xác định được dòng
                            ↪ phiếu mượn tương ứng, không thể hủy biên bản.");
163
164                   int rows = conn.Execute(
165                       @"UPDATE LoanSlipDetails SET ReturnedQuantity =
                            ↪ ReturnedQuantity - @qty
166                         WHERE DetailId = @detailId AND ReturnedQuantity >=
                            ↪ @qty",
167                       new { qty = line.Quantity, detailId }, tx);
168                   if (rows == 0) throw new BusinessException("Số lượng đã
                        ↪ trả của phiếu không khớp biên bản, không thể hủy.");
169               }
170
171               var perDevice = lines
172                   .Select(l => (l.DeviceId, b: Bucket(l.ConditionCode,
                        ↪ l.Quantity)))
173                   .GroupBy(x => x.DeviceId).OrderBy(g => g.Key);
174               foreach (var g in perDevice)
175               {
176                   int good = g.Sum(x => x.b.Good), damaged = g.Sum(x =>
                        ↪ x.b.Damaged), lost = g.Sum(x => x.b.Lost);
177                   int rows = conn.Execute(
178                       @"UPDATE Devices
179                         SET AvailableQuantity = AvailableQuantity - @good,
180                             DamagedQuantity   = DamagedQuantity - @damaged,
181                             LostQuantity      = LostQuantity - @lost
182                         WHERE DeviceId = @deviceId
183                           AND AvailableQuantity >= @good AND
                              ↪ DamagedQuantity >= @damaged AND LostQuantity
                              ↪ >= @lost",
184                       new { good, damaged, lost, deviceId = g.Key }, tx);
185                   if (rows == 0)
186                       throw new BusinessException(
187                           $"Không thể hủy: thiết bị \"{DeviceName(conn, tx,
                                ↪ g.Key)}\" đã được cho mượn lại hoặc đã xử lý
                                ↪ (sửa chữa) " +
188                           "sau lần trả này, tồn kho hiện tại không đủ để
                                ↪ hoàn tác.");
189               }
190
191               int updated = conn.Execute(
192                   @"UPDATE ReturnRecords
193                     SET IsCancelled = 1, CancelledBy = @userId, CancelledAt
                        ↪ = SYSDATETIME(), CancelReason = @reason
194                     WHERE ReturnRecordId = @returnRecordId AND IsCancelled
                        ↪ = 0",
```

```
                new { userId, reason, returnRecordId }, tx);
            if (updated == 0) throw new BusinessException("Biên bản này đã được hủy trước đó.");

            conn.Execute(RecomputeStatusSql, new { loanSlipId = slipId }, tx);
            return 0;
        });

    const string RecordSelect =
        @"SELECT r.ReturnRecordId, r.RecordCode, r.LoanSlipId, s.SlipCode, b.FullName AS BorrowerName,
                 r.ReturnDate, r.ReceivedBy, u.FullName AS ReceivedByName,
                 ISNULL(r.GiverName, N'') AS GiverName, ISNULL(r.Note, N'') AS Note,
                 r.IsCancelled, ISNULL(cu.FullName, N'') AS CancelledByName, r.CancelledAt,
                 ISNULL(r.CancelReason, N'') AS CancelReason,
                 ISNULL((SELECT SUM(d.Quantity) FROM ReturnRecordDetails d WHERE d.ReturnRecordId = r.ReturnRecordId), 0) AS TotalQuantity
          FROM ReturnRecords r
          JOIN LoanSlips s ON s.LoanSlipId = r.LoanSlipId
          JOIN Borrowers b ON b.BorrowerId = s.BorrowerId
          JOIN Users u ON u.UserId = r.ReceivedBy
          LEFT JOIN Users cu ON cu.UserId = r.CancelledBy ";

    public static List<ReturnRecordRow> Search(RecordFilter f)
    {
        string? kw = string.IsNullOrWhiteSpace(f.Keyword) ? null : f.Keyword.Trim();
        string? like = kw is null ? null : $"%{kw}%";

        using var conn = Db.Open();
        return conn.Query<ReturnRecordRow>(
            RecordSelect +
            @"WHERE (@like IS NULL OR r.RecordCode LIKE @like OR s.SlipCode LIKE @like OR b.FullName LIKE @like)
                AND (@fromDate IS NULL OR r.ReturnDate >= @fromDate)
                AND (@toExclusive IS NULL OR r.ReturnDate < @toExclusive)
                AND (@cancelled IS NULL OR r.IsCancelled = @cancelled)
                AND (@loanSlipId IS NULL OR r.LoanSlipId = @loanSlipId)
              ORDER BY r.ReturnRecordId DESC",
            new
            {
                like,
                fromDate = f.From?.Date,
                toExclusive = f.To?.Date.AddDays(1),
                cancelled = f.Cancelled,
                loanSlipId = f.LoanSlipId,
            }).ToList();
    }

    public static ReturnRecordFull? GetFull(int returnRecordId)
    {
        using var conn = Db.Open();
        var header = conn.QuerySingleOrDefault<ReturnRecordRow>(
```

```
            RecordSelect + "WHERE r.ReturnRecordId = @returnRecordId",
                ↪ new { returnRecordId });
        if (header is null) return null;
        return new ReturnRecordFull { Header = header, Details =
            ↪ GetDetails(conn, new[] { returnRecordId }) };
    }

    public static List<ReturnRecordFull> GetFullByLoan(SqlConnection
        ↪ conn, int loanSlipId)
    {
        var headers = conn.Query<ReturnRecordRow>(
            RecordSelect + "WHERE r.LoanSlipId = @loanSlipId ORDER BY
                ↪ r.ReturnDate, r.ReturnRecordId", new { loanSlipId
                ↪ }).ToList();
        if (headers.Count == 0) return new List<ReturnRecordFull>();

        var details = GetDetails(conn, headers.Select(h =>
            ↪ h.ReturnRecordId).ToArray());
        return headers.Select(h => new ReturnRecordFull
        {
            Header = h,
            Details = details.Where(d => d.ReturnRecordId ==
                ↪ h.ReturnRecordId).ToList(),
        }).ToList();
    }

    static List<ReturnRecordDetailRow> GetDetails(SqlConnection conn,
        ↪ int[] recordIds) =>
        conn.Query<ReturnRecordDetailRow>(
            @"SELECT d.Id, d.ReturnRecordId, d.DeviceId, dv.DeviceCode,
                ↪ dv.DeviceName, ISNULL(dv.Unit, N'') AS Unit,
                    d.Quantity, d.ConditionCode, ISNULL(d.Condition,
                        ↪ N'') AS Condition
              FROM ReturnRecordDetails d
              JOIN Devices dv ON dv.DeviceId = d.DeviceId
              WHERE d.ReturnRecordId IN @recordIds
              ORDER BY d.ReturnRecordId, d.Id", new { recordIds
                ↪ }).ToList();
}
```

Phụ lục J. Mã nguồn tiêu biểu – Tầng nghiệp vụ (Services)

Tệp `LoanService.cs`: kiểm tra quyền và dữ liệu đầu vào trước khi gọi tầng Data.

```
using QuanLyMuonTra.App.Data;
using QuanLyMuonTra.App.Models;
using QuanLyMuonTra.App.Security;

namespace QuanLyMuonTra.App.Services;

/// <summary>Nghiệp vụ phiếu mượn: F09 lập phiếu, F10 tra cứu, F11 chi
    ↪ tiết.</summary>
public static class LoanService
{
    public sealed record Created(int LoanSlipId, string SlipCode);
```

```
11
12      /// <summary>Kiểm tra dữ liệu phiếu mượn trước khi mở transaction.
        ↪  Trả về thông báo lỗi hoặc null nếu hợp lệ.</summary>
13      public static string? Validate(NewLoanSlip s)
14      {
15          if (s == null || s.Items == null) return "Dữ liệu phiếu mượn
            ↪  không hợp lệ.";
16          if (s.BorrowerId <= 0) return "Vui lòng chọn người mượn.";
17          if (s.BorrowDate.Date > DateTime.Today) return "Ngày mượn không
            ↪  được ở tương lai.";
18          if (s.DueDate.Date < s.BorrowDate.Date) return "Ngày dự kiến trả
            ↪  không được trước ngày mượn.";
19          if (s.Note is { Length: > 255 }) return "Ghi chú tối đa 255 ký
            ↪  tự.";
20          if (s.Items.Count == 0) return "Phiếu mượn phải có ít nhất một
            ↪  thiết bị.";
21          if (s.Items.Any(i => i.DeviceId <= 0)) return "Có dòng thiết bị
            ↪  không hợp lệ.";
22          if (s.Items.Any(i => i.Quantity <= 0)) return "Số lượng mượn của
            ↪  mỗi thiết bị phải lớn hơn 0.";
23          if (s.Items.Any(i => i.Quantity > 100000)) return "Số lượng mượn
            ↪  quá lớn.";
24          if (s.Items.GroupBy(i => i.DeviceId).Any(g => g.Count() > 1))
            ↪  return "Một thiết bị chỉ được xuất hiện một lần trong phiếu
            ↪  mượn.";
25          if (s.DueDate.Date > s.BorrowDate.Date.AddYears(1)) return "Hạn
            ↪  trả không được quá 1 năm kể từ ngày mượn.";
26          return null;
27      }
28
29      public static Result<Created> Create(NewLoanSlip slip) =>
30          Svc.Run(new[] { Perm.LoanCreate }, () =>
31          {
32              string? error = Validate(slip);
33              if (error != null) throw new BusinessException(error);
34              slip.Note = Svc.Clean(slip.Note);
35              var (id, code) = LoanRepository.Create(slip,
                ↪  Svc.CurrentUserId);
36              return new Created(id, code);
37          });
38
39      public static Result<List<LoanSlipRow>> Search(LoanFilter filter) =>
40          Svc.Run(new[] { Perm.LoanView }, () =>
            ↪  LoanRepository.Search(filter));
41
42      /// <summary>Phiếu còn thiết bị chưa trả, dùng cho màn hình ghi nhận
        ↪  trả (F12).</summary>
43      public static Result<List<LoanSlipRow>> SearchOpen(string? keyword)
        ↪  =>
44          Svc.Run(new[] { Perm.ReturnCreate, Perm.LoanView }, () =>
            ↪  LoanRepository.SearchOpen(keyword));
45
46      public static Result<LoanSlipFull> GetFull(int loanSlipId) =>
47          Svc.Run(new[] { Perm.LoanView, Perm.ReturnCreate,
            ↪  Perm.ReportPrint }, () =>
48              LoanRepository.GetFull(loanSlipId) ?? throw new
                ↪  BusinessException("Phiếu mượn không tồn tại."));
49
```

```
    public static Result<LoanSlipFull> GetFullByCode(string slipCode) =>
        Svc.Run(new[] { Perm.LoanView, Perm.ReturnCreate }, () =>
        {
            var header = LoanRepository.GetByCode(slipCode.Trim()) ??
            ↪  throw new BusinessException("Không tìm thấy phiếu
            ↪  mượn.");
            return LoanRepository.GetFull(header.LoanSlipId) ?? throw new
            ↪  BusinessException("Phiếu mượn không tồn tại.");
        });
}
```

```
using QuanLyMuonTra.App.Data;
using QuanLyMuonTra.App.Models;
using QuanLyMuonTra.App.Security;

namespace QuanLyMuonTra.App.Services;

/// <summary>Nghiệp vụ trả thiết bị: F12 ghi nhận trả, F13 biên bản, hủy
↪  biên bản.</summary>
public static class ReturnService
{
    public sealed record Created(int ReturnRecordId, string RecordCode);

    public static string? Validate(NewReturnRecord r)
    {
        if (r.LoanSlipId <= 0) return "Vui lòng chọn phiếu mượn.";
        if (r.ReturnDate.Date > DateTime.Today) return "Ngày trả không
        ↪  được ở tương lai.";
        if (r.Items.GroupBy(x => x.LoanSlipDetailId).Any(g => g.Count() >
        ↪  1)) return "Một dòng thiết bị chỉ được trả một lần trong biên
        ↪  bản.";
        if (r.Items.Count == 0) return "Chưa nhập số lượng trả cho thiết
        ↪  bị nào.";
        if (r.GiverName is { Length: > 100 }) return "Tên người giao tối
        ↪  đa 100 ký tự.";
        if (r.Note is { Length: > 255 }) return "Ghi chú tối đa 255 ký
        ↪  tự.";

        foreach (var i in r.Items)
        {
            if (i.LoanSlipDetailId <= 0) return "Có dòng thiết bị không
            ↪  hợp lệ.";
            if (i.Quantity <= 0) return "Số lượng trả phải lớn hơn 0.";
            if (!Enum.IsDefined(i.Condition)) return "Tình trạng thiết bị
            ↪  không hợp lệ.";
            if (i.ConditionNote is { Length: > 100 }) return "Ghi chú
            ↪  tình trạng tối đa 100 ký tự.";
            if (i.Condition is ReturnCondition.Damaged or
            ↪  ReturnCondition.MissingAccessory
                && string.IsNullOrWhiteSpace(i.ConditionNote))
                return $"Vui lòng ghi chú chi tiết khi thiết bị ở tình
                ↪  trạng \"{LoanText.Condition((int)i.Condition)}\".";
        }
        return null;
    }

    public static Result<Created> Create(NewReturnRecord record) =>
        Svc.Run(new[] { Perm.ReturnCreate }, () =>
```

```
        {
            string? error = Validate(record);
            if (error != null) throw new BusinessException(error);

            record.GiverName = Svc.Clean(record.GiverName);
            record.Note = Svc.Clean(record.Note);
            foreach (var i in record.Items) i.ConditionNote =
            ↪ Svc.Clean(i.ConditionNote);
            // Chỉ chọn ngày -> lấy giờ hiện tại nếu là hôm nay, ngược
            ↪ lại đặt 12:00
            if (record.ReturnDate.TimeOfDay == TimeSpan.Zero)
                record.ReturnDate = record.ReturnDate.Date ==
                ↪ DateTime.Today
                    ? DateTime.Now : record.ReturnDate.Date.AddHours(12);

            var (id, code) = ReturnRepository.Create(record,
            ↪ Svc.CurrentUserId);
            return new Created(id, code);
        });

    public static Result Cancel(int returnRecordId, string reason) =>
        Svc.Run(new[] { Perm.RecordCancel }, () =>
        {
            reason = (reason ?? "").Trim();
            if (reason.Length < 3) throw new BusinessException("Vui lòng
            ↪ nhập lý do hủy biên bản.");
            if (reason.Length > 255) throw new BusinessException("Lý do
            ↪ hủy tối đa 255 ký tự.");
            ReturnRepository.Cancel(returnRecordId, Svc.CurrentUserId,
            ↪ reason);
        });

    public static Result<List<ReturnRecordRow>> Search(RecordFilter
    ↪ filter) =>
        Svc.Run(new[] { Perm.RecordView }, () =>
        ↪ ReturnRepository.Search(filter));

    public static Result<ReturnRecordFull> GetFull(int returnRecordId) =>
        Svc.Run(new[] { Perm.RecordView, Perm.ReportPrint }, () =>
            ReturnRepository.GetFull(returnRecordId) ?? throw new
            ↪ BusinessException("Biên bản không tồn tại."));
}
```

Phụ lục K. Mã nguồn tiêu biểu – Băm mật khẩu và phiên đăng nhập

Tệp `Security/PasswordHasher.cs` (PBKDF2-SHA256) và `Security/Session.cs`.

```
using System.Security.Cryptography;

namespace QuanLyMuonTra.App.Security;

/// <summary>Băm mật khẩu bằng PBKDF2-SHA256. Định dạng lưu:
↪ PBKDF2$iterations$salt(base64)$hash(base64).</summary>
public static class PasswordHasher
{
```

```
    const int Iterations = 100_000;
    const int SaltSize = 16;
    const int KeySize = 32;

    public static string Hash(string password)
    {
        var salt = RandomNumberGenerator.GetBytes(SaltSize);
        var key = Rfc2898DeriveBytes.Pbkdf2(password, salt, Iterations,
        ↪ HashAlgorithmName.SHA256, KeySize);
        return $"PBKDF2${Iterations}${Convert.ToBase64String(salt)}${Con⌋
        ↪ vert.ToBase64String(key)}";
    }

    public static bool Verify(string password, string stored)
    {
        var parts = stored.Split('$');
        if (parts.Length != 4 || parts[0] != "PBKDF2" ||
        ↪ !int.TryParse(parts[1], out var iterations))
            return false;

        try
        {
            var salt = Convert.FromBase64String(parts[2]);
            var expected = Convert.FromBase64String(parts[3]);
            var actual = Rfc2898DeriveBytes.Pbkdf2(password, salt,
            ↪ iterations, HashAlgorithmName.SHA256, expected.Length);
            return CryptographicOperations.FixedTimeEquals(actual,
            ↪ expected);
        }
        catch (FormatException)
        {
            return false;
        }
    }
}
```

```
using QuanLyMuonTra.App.Models;

namespace QuanLyMuonTra.App.Security;

/// <summary>Phiên đăng nhập hiện tại: người dùng + tập quyền.</summary>
public static class Session
{
    static HashSet<string> _permissions =
    ↪ new(StringComparer.OrdinalIgnoreCase);

    public static AppUser? CurrentUser { get; private set; }

    public static void Start(AppUser user, IEnumerable<string>
    ↪ permissions)
    {
        CurrentUser = user;
        _permissions = new HashSet<string>(permissions,
        ↪ StringComparer.OrdinalIgnoreCase);
    }

    public static void End()
    {
```

```
20          CurrentUser = null;
21          _permissions = new
             ↪  HashSet<string>(StringComparer.OrdinalIgnoreCase);
22      }
23
24      public static bool Has(string permissionCode) =>
         ↪  _permissions.Contains(permissionCode);
25  }
```